Código Activo: IS-TEOR-PERI01
Nombre Activo: La Forensia como Herramienta en la Pericia Informática - REVISTA LATINOAMERICANA DE INGENIERIA DE SOFTWARE
Ver. Activo: 1.01 [14/9/16]
Copia de Versión Externa: Vol 1, No 6 (2013) - pp 237-240
Observaciones: Archivo descargado de http://sistemas.unla.edu.ar/sistemas/redisla/ReLAIS/ReLAIS-V1.htm en set/2016 (Vol 1, No 6 (2013): Tapa +articulo "La Forensia como Herramienta en la Pericia Informática")

REVISTA LATINOAMERICANA DE INGENIERIA DE SOFTWARE

DICIEMBRE 2013 VOLUMEN 1 NUMERO 6 ISSN 2314-2642

PUBLICADO POR EL GISI-UNLa

EN COOPERACIÓN POR LOS MIEMBROS DE LA RED DE INGENIERÍA DE SOFTWARE DE LATINOAMÉRICA

ARTÍCULOS TÉCNICOS

La Forensia como Herramienta en la Pericia Informática 237-240
Darío A. Piccirilli

Modelo de Sistema Basado en Conocimiento en el Dominio de la Seguridad de Aplicaciones 241-252
María Victoria Bajarlía, Jorge Ierache, Jorge Eterovic

Gestión y Desarrollo de Proyectos de Software: Metodología Ágil basada en Telecomunicaciones - MATe 253-258
Marisa Cecilia Tumino, Juan Manuel Bournissen, Claudio Bracalenti, Eric Schlemper, Silvio Kucharski

COMUNICACIONES SOBRE INVESTIGACIONES EN PROGRESO

Investigación en Progreso: Método de Evaluación Dinámica de Planes en Sistemas Inteligentes Autónomos 259-263
Ezequiel Gonzalez

REVISTA LATINAMERICANA DE INGENIERIA DE SOFTWARE

La Revista Latinoamericana de Ingenieria del Software es una publicación tecnica auspiciada por la Red de Ingeniería de Software de Latinoamérica (RedISLA). Busca: [a] difundir artículos técnicos, empíricos y teóricos, que resulten críticos en la actualización y fortalecimiento de la Ingeniería de Software Latinoamericana como disciplina y profesión; [b] dar a conocer de manera amplia y rápida los desarrollos científico-tecnológicos en la disciplina de la región latinoamericana; y [c] contribuir a la promoción de la cooperación institucional entre universidades y empresas de la Industria del Software. La línea editorial, sin ser excluyente, pone énfasis en sistemas no convencionales, entre los que se prevén: sistemas móviles, sistemas, plataformas virtuales de trabajo colaborativo, sistemas de explotación de información (minería de datos), sistemas basados en conocimiento, entre otros; a través del intercambio de información científico y tecnológica, conocimientos, experiencias y soluciones que contribuyan a mejorar la cadena de valor del desarrollo en la industria del software.

Comité Editorial

RAUL AGUILAR VERA (México)
Cuerpo Academico de Ingeniería de Software
Facultad de Matemáticas
Universidad Autonoma de Yucatán

PAOLA BRITOS (Argentina)
Grupo de Ingeniería de Explotación de Información
Laboratorio de Informática Aplicada
Universidad Nacional de Río Negro

RAMON GARCÍA-MARTINEZ (Argentina)
Grupo de Investigación en Sistemas de Información
Licenciatura en Sistemas
Departamento de Desarrollo Productivo y Tecnológico
Universidad Nacional de Lanús

ALEJANDRO HOSSIAN (Argentina)
Grupo de Investigación de Sistemas Inteligentes en Ingeniería
Facultad Regional del Neuquén
Universidad Tecnológica Nacional

BELL MANRIQUE LOSADA (Colombia)
Programa de Ingeniería de Sistemas
Facultad de Ingeniería
Universidad de Medellín

CLAUDIO MENESES VILLEGAS (Chile)
Departamento de Ingeniería de Sistemas y Computación
Facultad de Ingeniería y Ciencias Geológicas
Universidad Católica del Norte

JONÁS MONTILVA C. (Venezuela)
Facultad de Ingeniería
Escuela de Ingeniería de Sistemas
Universidad de Los Andes

MARÍA FLORENCIA POLLO-CATTANEO (Argentina)
Grupo de Estudio en Metodologías de Ingeniería de Software
Facultad Regional Buenos Aires
Universidad Tecnológica Nacional

JOSÉ ANTONIO POW-SANG (Perú)
Maestría en Informática
Pontifica Universidad Católica del Perú

DIEGO VALLESPIR (Uruguay)
Instituto de Computación
Universidad de la Republica

FABIO ALBERTO VARGAS AGUDELO (Colombia)
Dirección de Investigación
Tecnológico de Antioquia

CARLOS MARIO ZAPATA JARAMILLO (Colombia)
Departamento de Ciencias de la Computación y de la Decisión
Facultad de Minas
Universidad Nacional de Colombia

Contacto

Dirigir correspondencia electrónica a:
Editor de la Revista Latinoamericana de Ingenieria de Software
RAMON GARCIA-MARTINEZ
e-mail: rgarcia@unla.edu.ar
e-mail alternativo: rgm1960@yahoo.com
Página Web de la Revista: http://www.unla.edu.ar/sistemas/redisla/ReLAIS/index.htm
Página Web Institucional: http://www.unla.edu.ar

Dirigir correspondencia postal a:
Editor de la Revista Latinoamericana de Ingenieria de Software
RAMON GARCIA-MARTINEZ
Licenciatura en Sistemas. Departamento de Desarrollo Productivo y Tecnológico
Universidad Nacional de Lanus
Calle 29 de Septiembre No 3901. (1826) Remedios de Escalada, Lanús.
Provincia de Buenos Aires. ARGENTINA.

Normas para Autores

Cesión de Derechos de Autor

Los autores toman conocimiento y aceptan que al enviar una contribución a la Revista Latinoamericana de Ingenieria del Software, ceden los derechos de autor para su publicación electrónica y difusión via web por la Revista. Los demas derechos quedan en posesión de los Autores.

Políticas de revisión, evaluación y formato del envío de manuscritos

La Revista Latinoamericana de Ingenieria del Software recibe artículos inéditos, producto del trabajo de investigación y reflexión de investigadores en Ingenieria de Software. Los artículos deben presentarse redactados en el castellano en el formato editorial de la Revista. El incumplimiento del formato editorial en la presentación de un artículo es motivo de retiro del mismo del proceso de evaluación. Dado que es una publicación electrónica no se imponen limites sobre la cantidad de paginas de las contribuciones enviadas. También se pueden enviar comunicaciones cortas que den cuenta de resultados parciales de investigaciones en progreso.
Las contribuciones recibidas están sujetas a la evaluación de pares. Los pares que evaluaran cada contribución son seleccionados por el Comité Editorial. El autor que envíe la contribución al contacto de la Revista será considerado como el autor remitente y es con quien la revista manejará toda la correspondencia relativa al proceso de evaluación y posterior comunicación.
Del proceso de evaluación, el articulo enviado puede resultar ser: [a] aceptado, en cuyo caso será publicado en el siguiente numero de la revista, [b] aceptado con recomendaciones, en cuyo caso se enviará al autor remitente la lista de recomendaciones a ser atendidas en la nueva versión del articulo y su plazo de envio; ó [c] rechazado, en cuyo caso será devuelto al autor remitente fundando el motivo del rechazo.

Temática de los artículos

La Revista Latinoamericana de Ingeniería del Software busca artículos empíricos y teóricos que resulten críticos en la actualización y fortalecimiento de la Ingeniería de Software Latinoamericana como disciplina y profesión.
La línea editorial pone énfasis en sistemas no convencionales, entre los que se prevén: sistemas móviles, sistemas multimediales vinculados a la televisión digital, plataformas virtuales de trabajo colaborativo, sistemas de explotación de información (minería de datos), sistemas basados en conocimiento, entre otros. Se privilegiarán artículos que contribuyan a mejorar la cadena de valor del desarrollo en la industria del software.

Política de Acceso Abierto

La Revista Latinoamericana de Ingeniería de Software:

- Sostiene su compromiso con las políticas de Acceso Abierto a la información científica, al considerar que tanto las publicaciones científicas como las investigaciones financiadas con fondos públicos deben circular en Internet en forma libre, gratuita y sin restricciones.
- Ratifica el modelo de Acceso Abierto en el que los contenidos de las publicaciones científicas se encuentran disponibles a texto completo libre y gratuito en Internet, sin embargos temporales, y cuyos costos de producción editorial no son transferidos a los autores.
- No retiene los derechos de reproducción o copia (copyright), por lo que los autores podrán disponer de las versiones finales de publicación, para difundirlas en repositorios institucionales, blogs personales o cualquier otro medio electrónico, con la sola condición de hacer mención a la fuente original de publicación, en este caso Revista Latinoamericana de Ingeniería de Software.

Lista de Comprobación de Preparación de Envíos

Como parte del proceso de envío, se les requiere a los autores que indiquen que su envío cumpla con todos los siguientes elementos, y que acepten que envíos que no cumplan con estas indicaciones pueden ser devueltos al autor:

1. La contribución respeta el formato editorial de la Revista Latinoamericana de Ingenieria del Software (ver plantilla).
2. Actualidad/Tipo de referencias: El 45% de las referencias debe hacerse a trabajos de publicados en los últimos 5 años, así como a trabajos empíricos, para cualquier tipo de artículo (empírico o teórico).
3. Características artículos empíricos (análisis cuantitativo de datos): Se privilegian artículos empíricos con metodologías experimentales y cuasi experimentales, con pruebas estadísticas robustas y explicitación del cumplimiento de los supuestos de las pruebas estadísticas usadas.
4. Características artículos empíricos (análisis cualitativo de datos): Se privilegian artículos empíricos con estrategias de triangulación teórica-empírica, definición explícita de categorías orientadoras, modelo teórico de análisis, y análisis de datos asistido por computadora.
5. Características artículos de revisión: Para el caso de artículos de revisión, se evaluará que los mismos hayan sido desarrollados bajo el formato de meta análisis, la cantidad de referencias deben superar las 50, y deben explicitarse los criterios de búsqueda, bases consultadas y pertinencia disciplinar del artículo.
6. El artículo (en formato word y pdf) se enviará adjunto a un correo electrónico dirigido al contacto de la Revista en el que deberá constar la siguiente información: dirección postal del autor, dirección de correo electrónico para contacto editorial, número de teléfono y fax, declaración de que el artículo es original, no ha sido previamente publicado ni está siendo evaluado por otra revista o publicación. También se debe informar de la existencia de otras publicaciones similares escritas por el autor y mencionar la versión de la aplicación de los archivos remitidos (versión del editor de texto y del conversor a archivo pdf) para la publicación digital del artículo.
7. Loa Autores aceptan que el no cumplimiento de los puntos anteriores es causal de No Evaluación del articulo enviado.

Compromiso de los Autores de Artículos Aceptados

La Revista Latinoamericana de Ingeniería del Software busca ser una revista técnica de calidad, en cuyo desarrollo estén involucrados los investigadores y profesionales latinoamericanos de la disciplina. En este contexto, los Autores de artículos aceptados asumen ante la Revista y la Comunidad Latinoamericana de Ingeniería del Software el compromiso de desempeñarse como pares evaluadores de nuevas contribuciones.

La Forensia como Herramienta en la Pericia Informática

Darío A. Piccirilli
Facultad Regional Buenos Aires. Universidad Tecnológica Nacional
Facultad de Informática. Universidad Nacional de La Plata
dpiccirilli@unlp.edu.ar, dariopiccirilli@gmail.com

—Abstract—Este documento contiene una descripción de los aportes que realiza la Forensia en Informática, contemplando las nuevas tecnologías que hoy día aplican en los procesos judiciales y que derivan en Pericias muy específicas y complicadas, tareas técnicas sobre las que no se puede generar ninguna duda en el tratamiento de la prueba. Es decir, el proceso de generación de la prueba, desde el secuestro de la misma hasta el análisis pericial, debe ser indubitable, de manera tal que quien deba impartir justicia pueda contar con elementos claros, contundentes y útiles. La informática puede considerarse que se encuentra relacionada en forma transversal con gran parte de la problemática judicial, aplicando en los distintos fueros de la Justica Argentina, tanto en lo Laboral, Comercial, Civil, Contencioso Administrativo Federal, Penal Económico, Criminal y también para la Corte Suprema.

***Palabras Clave*—Forensia, Pericia Infromática**

I. INTRODUCCION

Hoy día la informática se ha convertido en una herramienta o vehículo para la comisión de un delito., pero también esta ciencia es objeto de delitos. Dicho de otra manera, hoy es posible aplicar la informática para realizar una estafa, enviar una amenaza (intentando quedar en el anonimato), para obtener claves secretas de cuentas bancarias y así conseguir ilegalmente fondos dinerarios de otras personas, para realizar robo de datos de una empresa con distintos fines, acceso indebido a la información de la cía., daños en las páginas WEB, violación de la confidencialidad y secretos de la cía., entre otros.

A esto debemos sumarle casos vinculados con delitos sociales como la pedofilia o delitos federales como el Lavado de Activos y demás delitos financieros.

Lo expuesto hace que cuando se tenga que realizar una pericia informática, se deban considerar distintos aspectos a saber:

- El perfil del problema o del delito a peritar
- El procedimiento científico a aplicar
- La presencia de peritos de parte
- El procedimiento protocolar a aplicar, en relación a la situación procesal
- Las herramientas de forensia informática a aplicar o la combinación de más de una herramienta
- La posibilidad de nuevas pruebas
- La posibilidad de aclarar los puntos de pericia requeridos por el Juez que interviene en la causa
- La existencia de cadena de custodia de la prueba informática
- Las condiciones en que la prueba ha sido preservada

Sobre la base de lo mencionado, es de destacar que la forensia informática es un componente muy importante dentro de las Pericias Informáticas. No obstante lo expuesto, existe una novedosa aplicación de las herramientas de forensia para situaciones privadas, para pre constituír pruebas en forma previa a un pleito. Esto se aplica básicamente en los fueros Civil, Laboral o Comercial. Pues en el fuero Penal, generalmente la prueba ocurre ante un secuestro.

II. DESARROLLO

Considerando lo planteado en la introducción al tema, se desarrollan a continuación los puntos que caracterizan la temática planteada.

A. Perfil del problema o del delito a peritar

Es necesario saber que nunca existe una pericia informática igual a otra, a pesar que se encuentren caracterizadas por el mismo tipo de delito. Siempre varía:

- el escenario
- las pruebas y la modalidad en que han sido obtenidas
- las partes que intervienen en el pleito o en la investigación
- las herramientas que deben usarse
- el conocimiento que el perito debe aplicar, como así también su experiencia

Por ello, a veces es necesario integrar más de un perito a la tarea pericial. Pues a pesar que tengan todos la misma especificidad, es muy probable que no todos cuenten con la misma especialidad, experiencia y grado d conocimiento sobre la tarea a realizar.

B. Procedimiento científico a aplicar

Es el protocolo que se debe aplicar al momento de realizar una pericia informática, y parte del mismo es la herramienta de forensia a utilizar.

En el caso de existir peritos de parte, esta tarea debe ser consensuada entre todos los participantes, con el objetivo de obtener una prueba clara y útil para impartir justicia por la autoridad competente.

La tarea pericial debe ser debidamente comunicada a los especialistas de parte, debido a que la tarea no puede ser realizada en forma exclusiva por peritos de oficio, existiendo técnicos de parte que deban participar.

C. Herramientas de Forensia

Conforme lo establece el FBI, "*la informática (o computación) forense es la ciencia de adquirir, preservar, obtener y presentar datos que han sido procesados electrónicamente y guardados en un medio computacional"*. Por lo tanto, al momento de seleccionar es necesario conocer claramente lo que el Juez requiere para completar su investigación. Pues una investigación forense puede integrarse de las siguientes etapas que se describen en la siguientes subsecciones.

C.1. Objetivo a cumplir – investigar durante la pericia

En primer lugar se debe analizar el dispositivo objeto de la pericia. Por ejemplo: si es un disco rígido o un teléfono celular. En el caso de un disco rígido, es posible que se necesite investigar sobre:

- las características del sistema operativo instalado en el disco rígido
- si existen archivos borrados en un disco rígido secuestrado
- verificar las fechas de creación, modificación, borrado o último acceso a os archivos
- buscar archivos en espacios liberados por el sistema operativo que posee el disco rígido
- analizar si los discos fueron cambiados
- buscar en archivos que poseen claves y que no han sido provistas en forma previa al análisis de la información

En el caso de un teléfono celular, es posible que se necesite investigar sobre:

- mensajes de texto enviados y recibidos
- correos electrónicos enviados y recibidos
- contactos guardados en el dispositivo
- imágenes
- mensaje de voz
- georeferenciación de los archivos

Es de señalar que no todos las marcas y modelos de teléfonos celulares tienen las mismas posibilidades de búsqueda y hallazgos, particularmente los de origen chino, que en algunos casos llegan a poseer hasta seis (6) chips.

C.2. Protocolo a seguir una vez clarificado el objetivo

Una vez evaluado el dispositivo celular o elemento óptico o magnético a estudiar, es necesario definir:

- si se va a aplicar sobre un teléfono celular, se debe analizar sobre las herramientas de forensia que se disponen, si se aplican aquellas que se encuentran con hardware y software integrado o aquellas que se aplican a través de software solamente
- si se va aplicar sobre elementos magnéticos – comunes o de estado sólido (por ejemplo pendrives, discos rígidos), se debe aplicar duplicadores forenses, que permiten generar una copia imagen del disco origen, sin alterarlo. También se aplican elementos de protección contra escritura o bloqueadores, con el objetivo de evitar contaminar la prueba. Esta protección de escritura puede ser por hardware o software.
- existe la posibilidad de tener que realizar la forensia informática in situ, es decir en el momento del allanamiento. Para ello existen soluciones de herramientas "portables", que permiten obtener pruebas bajo protocolos de forensia, sin contaminar la prueba y sin retirar o secuestrar los elementos del lugar que se allana.

Esto permite aplicar criterios prácticos al momento del procedimiento del pedido de secuestro, pues existen situaciones que el parque de computadoras que se investiga es muy grande (por ejemplo, mas 50 equipos), y sería muy complicado retirar todos los elementos, cuando se sospecha que solamente en alguno de ellos puede existir información de interés para la causa que se investiga.

C.3. Aspectos a considerar una vez obtenida la evidencia digital

Es fundamental evaluar la forma en que se le va a entregar la evidencia digital obtenida. Es decir, como el propio FBI plantea cuando expresa que se debe considerar la forma de "presentar" dicha evidencia, se hace clara referencia a que una vez obtenidos los hallazgos, es importante evaluar la manera de estructurar los mismos para que el Juez pueda entenderlos y considerarlos. Pues normalmente las herramientas de forensia informática producen reportes que no son intuitivos de entender por alguien que no trabaja a diario con estos elementos. Si esta parte no se respeta, la pericia puede haber sido muy bien hecha, pero no le sirve de mucho al Juez, si no la puede entender bien.

Por ello, muchas veces se solicita separar los mensajes de texto de las imágenes, de los documentos en Word o de las planillas en Excel. También, que los correos electrónicos sean enviados en una interfase de fácil comprensión para el administrador de justicia. En muchos casos se aplica el "mailnavigator".

Se recomienda que, de ser posible y según la problemática que se investigue, se apliquen mas de una herramienta de forensia, combinando la potencia que cada una pueda tener. Por ejemplo, para copia de forensia al duplicar un disco rígido, se puede usar un dispositivo de un determinado proveedor y luego para realizar las búsquedas sobre la evidencia digital generada, se puede aplicar uan herramienta que pertenezca a otro proveedor.

Hoy día, existen en el mercado varios proveedores reconocidos de herramientas de forensia informática, como ser Guidance, Cellebrite, Access Data, a los que debe sumársele varios productos del estilo "free" o libres de licenciamiento, y por supuesto sin costo alguno. Es de aclarar que el hecho de no tener costo, no sacrifica la calidad del producto.

Teniendo en cuenta el amplio mercado que existe sobre herramientas de forensia informática, es de aclarar que no es que una sea mejor que otra, todo depende del escenario que se debe investigar y del dominio o experiencia que se tenga sobre cada una de ellas.

Como resultado de las tareas de investigación, en algunas situaciones, surge la necesidad de requerir nuevas pruebas digitales. En estos casos, el perito cuenta con la posibilidad de relevar bien el escenario en que se encuentran esas nuevas evidencias, y por ende, puede prepararse mejor para la obtención de las mismas.

Al momento de ordenar una pericia informática, existe la posibilidad o tal vez la necesidad de aclarar los puntos de pericia requeridos por el Juez. Ello debido a que muchas veces se le pide al perito la búsqueda de evidencia digital en forma

amplia o genérica. Es decir, no se definen por ejemplo las "palabras clave" sobre las que se debe realizar el análisis digital. Pues para realizar esta tarea, sobre la copia o imagen forense obtenida anteriormente, es necesario ser muy preciso en lo que se debe analizar, ya que guarda directa relación sobre el objeto. Por ejemplo, se puede especificar el número de una cuenta bancaria, una dirección de IP, la especificación de un correo electrónico, la especificación de una imagen, etc.

Esto es fundamental que se encuentre especificado en los puntos de pericia, pues de lo contrario puede quedar librado al criterio del perito que realiza la tarea, y probablemente dicho criterio puede no coincidir con la estrategia de investigación que se sigue.

C.4. Cadena de custodia de la prueba informática

Se debe analizar su existencia. Este es un procedimiento que muy pocas veces se aplica y que debe originarse en el primer contacto con la evidencia digital, generalmente durante el secuestro. Esta parte del protocolo es considerada como una simple formalidad, pero en realidad es de vital importancia llevar una especie de "historia clínica" de todos los pasos que se siguen con dicha evidencia.

No debemos olvidar que muchas veces del momento que se accede a la prueba o evidencia digital hasta el momento que llega al perito informático, no sólo pasa un considerable tiempo, sino que pasa por distintas etapas (del allanamiento a la comisaría, de allí al juzgado, de allí al perito).

D. Condiciones en que la prueba ha sido preservada

Al momento de recibir los elementos a peritar, debe verificar si ha sido resguardada con franjas de secuestro, si ha sido sellada en todos sus puertos de acceso, si viene en bolsas de nylon transparentes o de color, si viene en cajas de cartón, si tienen protección en papel especial contra golpes, si vienen resguardados en protecciones de espuma anti estática, entre otros aspectos.

En otro orden conceptual, y al sólo efecto ilustrativo de la potencialidad y variedad de herramientas que hoy día existen en e mercado, es de señalar a modo informativo algunas de ellas:

Disk Jockey: se utiliza para la duplicación de discos en paralelo

Tableau Dead Collection: se utiliza para prevenir la escritura sobre aquellos dispositivos de almacenamiento que fueron secuestrados o aportados como prueba

EnCase: posee varios módulos de aplicación, entre ellosla realización de una copia forense o imagen de discos o dispositivos mganeticos, ya sea desde un disco secuestrado o en el momento de un allanamiento (conocido como Dead/Live Collection)

Linen CrossOver Acquisition: permite realizar copias de información almacenada en dispositivos mganético, de manera segura y en vivo

F-Response - Network Acquisition - Write Blocker Dead Collection: permite la protección de escritura sobre dispositivos con conexión del tipo USB

Zero View: permite leer las cabeceras de los discos rígidos

FTK: permite realizar copias imagen de dispositivos magnéticos secuestrados o en el momento del allanamiento (módlo conocido como Dead/ive Collection)

Cellebrite – UFED: permite realizar un análisis forense del contenido de una gran variedad de teléfonos celulares sobre todos los modelos existentes en el mercado internacional. Incluye gran variedad de teléfonos de origen chino, ue generalmente son muy clejos de estudiar. Esta herramienta permite identificar la información contenida en el dispositivo, incluyendo mensajes entrantes – salientes, lista de contactos, llamadas entrantes – salientes, correos electrónicos entrantes = salientes.

III. Conclusiones Preliminares

A modo de reflexión preliminar, es de señalar que cada herramienta cumple una finalidad específica con más eficiencia que otras. Por ejemplo, existen herramientas que son muy intuitivas para realizar búsqueda información sobre palabras clave que generalmente ordena un Juez, pero no son tan efectivas en el bloqueo de escritura por hardware. Otras que permiten un excelente bloqueo de escritura para preservar la prueba de contaminación digital, pero no son ágiles para búsqueda y análisis sore pañabras clave.

Generalmente, y dependiendo del problema a analizar, se sugiere aplicar una combinación de herramientas, para asegurar la efectividad que se debe tener en estos casos donde la libertad de las personas puede estar comprometida, y ello podría depender del resultado de una pericia informática aplicando herramientas de forensia.

Referencias

[1] Código Procesal Penal de la Nación Argentina. http://www.infojus.gov.ar

[2] Código Procesal Civil de la Nación Argentina. http://www.infojus.gov.ar

[3] Argentina: Ley 24.766 de Confidencialidad (30/12/1996) http://www.infojus.gov.ar

[4] Argentina: Ley 24.769 Penal Tributaria (15/01/1997). http://www.infojus.gov.ar

[5] Argentina: Ley 25.036 Propiedad. Intelectual (15/11/1998) http://www.infojus.gov.ar

[6] Argentina: Ley 25.236 Habeas Data (02/11/2000) http://www.infojus.gov.ar

[7] Alvarado Lemus, J. 2008. "Cibercrimen", Artemis Edinter, De Museo,

[8] [Mikel Gatesi – Dani Creus, "El cibercrimen y las guerras de robots: Search & Destroy", 2012

[9] Phil Williams, "Crimen Organizado y Cibernético" Centro de Enseñanza en Seguridad de la Internet de la Universidad Carnegie Mellon, 2008

[10] Argentina: Ley 25.891 Servicio de Comunicaciones Móviles (25/05/2004) http://www.infojus.gov.ar

[11] Argentina: Ley 25.930 Modificación Código Penal / Incluye Inc. 15 Art. 173 y Modificación Art. 285 http://www.infojus.gov.ar

[12] [12] Argentina: Artículo 44 Código Contravencional de la Ciudad Autónoma de Buenos Aires

http://www.infojus.gov.ar

[13] Argentina: Ley 26.388 Delitos Informáticos (24/06/2008) http://www.infojus.gov.ar

[14] Wilson 2003,Taylor, R., Caeti, T., Kall Loper, D., Fritsch, E y Liederbach, J. 2006

[15] Forensic Toolkit http://www.accessdata.com/products/utk

[16] WinHex http://www.x-ways.net/forensics/index-m.html

[17] [White,D., Rea, A., Mckenzie, B y Glorfled, L 2004, Cano 2006

[18] US Department of Justice, Electronic Crime Scene Investigation: “A Guide for First Responders”, 2001

[19] Fernando Miró Linares: “El Cibercrimen: Fenomenología y Criminología de la Delincuencia en el Ciberespacio”, Marcial Pons Ediciones Jurídicas y Sociales S.A., 1ra. Edición, 12/2012

[20] Misha Glenny. “El lado oscuro de la Red, el CiberCrimen, la Ciberguerra y TU, 2013

[21] Rina Begum, “Connect More”, KPMG Infrastructure Survey 2013, KPM LLC, 09/201



Dario Piccirilli. Licenciado en Sistemas de la Universidad Tecnológica Nacional (1979), con estudios de Posgrado como magister en Ingeniería de Software en el Instituto Tecnológico de Buenos Aires (Argentina) y Master en Ingeniería de Software en la Universidad Politécnica de Madrid (España). Profesor Titular en la Cátedra de Pericias Informáticas (Carrera de grado en Ingeniería en Sistemas de Información) y Profesor Titular en la Cátedra Auditoría, Seguridad y Pericias Informáticas (Carrera de Maestría en Sistemas de Información) en la Facultad Regional Buenos aires de la Universidad Tecnológica Nacional. Profesor Titular en Pericias Informáticas (Especialización de Posgrado en Redes) En la Facultad de Informática de la Universidad Nacional de La Plata (Argentina). Especialista en Pericias Informáticas en fueros Penal, Civil, Comercial y Laboral del Poder Judicial de la Nación de la Republica Argentina (desde 1989 a la fecha).

Modelo de Sistema Basado en Conocimiento en el Dominio de la Seguridad de Aplicaciones

María Victoria Bajarlía[1], Jorge Ierache[2,3], Jorge Eterovic[4]
1. Maestría en Ingeniería de Sistemas de Información. Universidad Tecnológica Nacional (FRBA)
2. Laboratorio de Sistemas Avanzados de Información Facultad de Ingeniería. Universidad de Buenos Aires
3. Instituto de Sistemas Inteligentes y Enseñanza Experimental de la Robótica FICCTE-Universidad de Morón
4. Universidad de Morón - Morón, Argentina
mvbajarlia@gmail.com, jierache@yahoo.com, jeterovic@unimoron.edu.ar

***Resumen*—El objetivo es proponer un modelo de un sistema basado en conocimiento (SBC) aplicado al análisis de seguridad de aplicaciones de gestión. El modelo se fundamenta en un sistema basado en conocimiento (SBC) que cuenta con un componente cognitivo que le permite incorporar conocimiento. En virtud de que las amenazas y los ataques informáticos representan un problema constante y creciente se puede suponer que el SBC, a través del aprendizaje dinámico que lo mantendrá actualizado, podrá asistir a los especialistas en Seguridad de la Información, en el área de competencia, a la elaboración de Especificación de Requerimientos.**

***Palabras Clave* —Seguridad de aplicaciones, sistemas basados en conocimiento.**

I. INTRODUCCIÓN

Se propone la aplicación de un sistema basado en conocimiento (SBC) aplicado al análisis de la seguridad de aplicaciones. La base de conocimiento será alimentada permanentemente por normas, estándares y mejores prácticas vigentes de la industria informática así como por aquellos informes de vulnerabilidades y ataques que tomen conocimiento público en la comunidad informática. El motor de inferencia, que trabajará sobre un universo abierto, tomará la información suministrada por la base de conocimiento para analizar la seguridad de una aplicación determinada. La solución del problema abarcará desde el análisis de seguridad de aplicaciones de gestión hasta el control de que las mismas cumplan con el marco regulatorio.

El presente trabajo es el producto resultante de un trabajo de investigación realizado en el marco de la tesis de la Maestría en Ingeniería en Sistemas de Información y pretende ser una contribución con la Seguridad de la Información.

A. El problema

El avance tecnológico y el desarrollo de aplicaciones informáticas para soportar las necesidades del negocio de una organización hace necesario traspasar fronteras en un contexto de infraestructura tecnológica, por ejemplo acceder desde la Web hasta llegar a una base de datos que está gestionada por un software que se ejecuta sobre un equipo Mainframe. De este modo la explotación de la aplicación se realiza atravesando diversas capas e integrando diferentes plataformas

existentes en la organización. Dado que las capas tienen distintas naturalezas de seguridad, es imperioso implementar un mecanismo eficiente que permita que las aplicaciones sean realmente seguras cumpliendo con los estándares respectivos en materia de Seguridad de la Información y permaneciendo altamente alineadas con la tecnología. [4] [8] [13] [16] [17] [18]

Por este motivo para abordar esta problemática se plantea un SBC que asista a la elaboración de especificaciones de requerimientos de software (ERS) a fin de colaborar con el desarrollo de aplicaciones seguras que contribuyan eficientemente a reducir las potenciales vulnerabilidades de las mismas. A su vez que permita evaluar si una aplicación dada se ajusta satisfactoriamente a los niveles de seguridad establecidos, contribuyendo con el mantenimiento y el refinamiento del conocimiento.

B. Áreas involucradas en el dominio del problema de estudio

Las áreas que participan en el contexto del tema propuesto involucran:

a) Seguridad de la Información (SI). Engloba la investigación del área de la seguridad de aplicaciones de gestión. [23] [26] [27]

b) Ingeniería de Requerimientos (IR). Se basa inicialmente en el estándar IEEE-830 de Especificación de Requisitos de Software (ERS), sobre el cual se realizan aportes en función del modelo de conocimiento que se obtenga del trabajo con los expertos en el área de Seguridad de la Información, a partir de las consideraciones que surjan en relación a requerimientos funcionales y no funcionales. [24] [28]

c) Ingeniería de Conocimiento (INCO). Incorpora el marco metodológico y las técnicas aplicadas al desarrollo de un SBC en el contexto dado de la INCO. Quedará comprendido por la extracción y educción de conocimientos con los Expertos del área de seguridad. [19] [25]

d) Sistema basado en conocimiento (SBC). Comprende la implementación de un sistema que asista a la elaboración de especificaciones en materia de seguridad de la aplicación en el marco de una ERS, a través de la incorporación de los aspectos de la materia de estudio en el modelo de conocimiento del Experto de campo. Por último, y como conclusión, abarcará la implementación de un prototipo de SBC para el análisis y evaluación de ERS en los aspectos de seguridad, desde el punto de vista de la IR. [2] [3] [4]

Es importante señalar el mecanismo de interacción de las áreas involucradas constituyendo el SBC:

- IR-SI. Aporta la base metodológica para construir las Especificaciones de Requerimientos de Software de

Seguridad en el aspecto específico de Seguridad de la Información (ERSSI) según el estándar IEEE-830.

- IR-INCO. Aporta la metodología para el desarrollo del SBC en el contexto de la IR.
- SI-INCO. Aporta la conceptualización como producto de la extracción de conocimiento (marco regulatorio, mejores prácticas, etc.) y la educción de conocimiento (entrevistas con el Experto y trabajo de campo), para la formalización e implementación del SBC.
- ERSSI-SBC. Como resultado de la interacción de las áreas involucradas se desarrolla un modelo que permita evaluar si una aplicación dada se ajusta a los niveles de seguridad establecidos, luego el especialista en seguridad de aplicaciones alimentará a la ERSSI a partir de nuevas regulaciones de la industria, mejores prácticas, etc. lo que contribuirá con el crecimiento del sistema. Por último dará lugar al mantenimiento del conocimiento y servirá de soporte para la construcción y refinamiento/mejora continua de ERSSI sobre la base del SBC. La Figura 1 sintetiza el mecanismo de interacción.

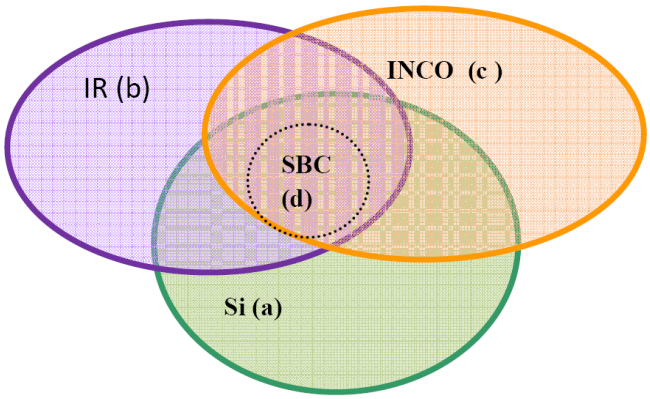


Fig. 1. Representación conceptual de las áreas involucradas

C. Estado del arte de seguridad de la información

La situación actual demuestra que si bien existe un importante nivel de madurez en materia de seguridad informática respecto de la infraestructura tecnológica organizacional no sucede lo mismo con los sistemas aplicativos que son soportados por dicha infraestructura. Esto conlleva a una falta de alineación entre analistas, constructores, arquitectos de software y especialistas en el análisis de vulnerabilidades en el software de gestión. Finalmente esta falta de alineación puede poner en riesgo uno de los activos más importantes que tiene una organización: su información.

La infraestructura de capas propuesta comprende: Autenticación, Servidor de Aplicaciones, Programas ejecutables y Repositorio de Datos. Como una evaluación preliminar del problema se realizo una extracción y educción de Expertos de conocimiento. Esto significa, en primer lugar, evaluar el tipo de seguridad que corresponde aplicar en cada una de las capas que componen el desarrollo de un software. En segundo lugar evaluar el trabajo de un Experto en esta materia a fin de extraer el conocimiento necesario en relación a las posibles vulnerabilidades de software que pueden surgir con el crecimiento tecnológico. [1] [9] [10] [11] [14] [19] [22] [29]

A continuación se enumeran resumidamente, para cada capa, los distintos aspectos investigados y que sirvieron como piedra fundamental para el desarrollo de la solución.

- Autenticación

-Mecanismo de autenticación (Mecanismos de autenticación estándares, etc.)

-Protección de la red interna (firewall y sus distintos tipos, etc.)

- Servidor de aplicaciones

- S.O.A (Arquitectura orientada a servicios).

-Servicios Web (protocolos estándares, transmisión de datos seguros de extremo a extremo, etc.)

- Programas ejecutables

-Aplicaciones Web y sus vulnerabilidades (mejores prácticas de la industria, ataques más comunes como ingeniería social y phishing entre otros)

-Seguridad de Programas Ejecutables (Auditoría de Código, Log de aplicaciones, Validaciones de datos de Entrada y de Salida, tratamiento de datos dentro del programa, etc.)

-Calidad de los datos (procedimientos de corrección, validación y normalización de datos, etc,)

-Gestión de liberación y reguardo de versiones fuente de programas

- Repositorio de datos

-Procedimientos asociados (Resguardo y recupero, Clasificación, etc.)

-Seguridad de Bases de datos (mejores prácticas recomendadas en la industria)

-Tecnologías para almacenamiento seguro (cifrado de datos, etc.)

II. Metodología de Desarrollo

Para la construcción del modelo se seleccionó una metodología adecuada del área de Ingeniería de Conocimiento (INCO): Metodología IDEAL. El desarrollo se articuló considerando las fases I y II de la metodología IDEAL en virtud de que permiten alcanzar el estado de un prototipo para la explotación de los conocimientos basales del dominio en cuestión. Dando como resultado productos como el Diccionario de Conceptos, la Tabla Concepto-Atributo-Valor, el Modelo de Entidad y Relación, el Mapa de Conocimiento, la Formalización en Marcos, la Base de Hechos, la Base de Reglas y el Motor de Inferencias.

Dentro de la Fase I (Identificación de la tarea) se consideran los objetivos principales para la construcción del Sistema Experto (SE), aplicado al problema a resolver, significa en primer lugar adquirir el conocimiento necesario en lo referente al marco regulatorio y mejores prácticas de la industria informática para la Seguridad de la Información. En segundo lugar hacer educción de los conocimientos de los especialistas en esta materia evaluando el trabajo de un Experto en Seguridad de la Información. Durante la Fase II (Desarrollo del prototipo) se continuó con las actividades de adquisición del conocimiento, se realizó la viabilidad del sistema, la conceptualización y la formalización de los conocimientos e implementación del prototipo que permitió validar con el Experto el modelo de SBC propuesto. A continuación se expone una síntesis de los ítems más significativos de la Fase II: Conceptualización, Formalización e Implementación.

A. Conceptualización del Conocimiento

La conceptualización comprende la identificación y adquisición de conocimientos Fácticos, Estratégicos y Tácticos, a fin de llegar a un Mapa de Conocimientos. Dicho mapa será la síntesis de la conceptualización de un Modelo

Dinámico y de un Modelo Estático que constituirán el modelo conceptual del SBC.

Dentro del Modelo Estático se trabaja en primer lugar con los conocimientos fácticos (Diccionario de Conceptos, Glosario de Términos, Tabla Concepto-Atributo-Valor y Modelo de Entidad-Relación, este último también denominado DER). Para ello se definieron los conceptos, sus atributos y valores asociados, así como las relaciones entre ellos, a partir de los conocimientos adquiridos y la educción de Expertos en materia de Seguridad de la Información. En segundo lugar se consideran los conocimientos estratégicos, a través de la identificación de funciones y actividades del proceso de resolución, análisis y juicio del Experto y la efectiva aplicación de mejores prácticas y estándares de Seguridad de la Información.

1) Conocimientos Fácticos

La propuesta inicial para la conceptualización incorpora la interacción del paradigma del conocimiento para ser instrumentado por el SBC, sobre las bases del paradigma funcional, considerando dentro de este último los requerimientos funcionales (RF) y los requerimientos no funcionales (RNF) desarrollados en la ERSSI. En la misma se presentan los requerimientos de Seguridad de la Información relacionados específicamente con la seguridad de las aplicaciones informáticas, en el dominio de las aplicaciones de gestión. Los requerimientos no funcionales quedan representados por características vinculadas con la configuración, la administración y el mantenimiento de un entorno seguro para las aplicaciones, dentro y fuera de la organización.

Dentro de la ERSSI también se definen niveles de aceptación a los que debe ajustarse una aplicación. Para la definición de los mismos se tomó como base la probabilidad de ocurrencia de amenazas y vulnerabilidades que puedan sufrir las aplicaciones así como el impacto que el riesgo asociado a las mismas puede causar en la organización. A modo de recomendación se clasifican en:

- Mandatario. Por considerar que su cumplimiento está fuertemente ligado al nivel de seguridad que requieren las aplicaciones en relación al dominio de la misma dentro de los objetivos de negocio de la organización.
- Sugerido. Por considerar que el no cumplimiento se relaciona con riesgos de mediana probabilidad de ocurrencia en el uso de las aplicaciones, desde el punto de vista de la seguridad de la información.
- Deseable. Por considerar que el no cumplimiento se relaciona con riesgos de baja probabilidad de ocurrencia en el uso de las aplicaciones.

El análisis inicial dentro del proceso de conceptualización es la evaluación del tipo de seguridad que corresponde aplicar en cada una de las capas que componen el desarrollo de un software y que forman parte de un contexto donde se desarrollarán las aplicaciones. En segundo lugar evaluar el trabajo de un Experto en Seguridad de la Información considerando todos los aportes que un especialista puede incorporar en dicha materia, completando así los puntos que nacen de la extracción de conocimiento. Posteriormente, la educción de conocimiento, producto de las heurísticas de los especialistas, permitió a través de distintas entrevistas ampliar el horizonte para luego mejorar cada aspecto con los aportes de los Expertos, la alineación a estándares relacionados a la Seguridad de la Información, la implementación de buenas prácticas, etc.

Es importante destacar que el contexto de aplicación está dado por una infraestructura de capas, de ella se desprenden los conceptos necesarios a fin de llegar al proceso de conceptualización.

Los conceptos se encuentran altamente vinculados con los requerimientos funcionales y no funcionales detallados en la ERSSI. La infraestructura de capas propuesta, comprende:

- Autenticación.
- Servidor de Aplicaciones
- Programas ejecutables
- Repositorio de Datos.

El segundo paso en el marco del proceso consiste en identificar las relaciones entre los conceptos definidos. Se trabaja con conocimientos fácticos y se busca simbolizar el modelo mental que el Experto tiene de la vista estática del problema a resolver a través de la observación y de las distintas entrevistas que se mantiene con el Experto a lo largo del trabajo, El modelo mental quedará reflejado a través del Modelo de Entidad-Relación (DER). Bajo la óptica de este modelo, el contexto de aplicación está conformado por las cuatro capas mencionadas anteriormente, para cada una de ellas se establecerá un subdiagnóstico que posteriormente aportará a la construcción del diagnóstico final que brindará el SBC. La relación entre los conceptos y los requerimientos de tipo funcional y no funcional quedará determinada por las contribuciones de la ERSSI y por los resultados logrados en función del diagnóstico que brindará el Experto y los aportes en materia de Seguridad de la Información que realicen los especialistas.

El modelo plantea como entidad principal el Diagnostico Final de Situación de Seguridad - DFSS que se relaciona con 4 entidades representadas por Subdiagnósticos (Capa Autenticación, Capa Servidor Aplicaciones, Capa Programas Ejecutables y Capa Repositorio de Datos). Cada subdiagnóstico tendrá relaciones de tipo “muchos a muchos” con entidades de tipo Conceptos según su propio contexto, por ejemplo Conceptos Capa Autenticación, Conceptos Capa Servidor Aplicaciones, Conceptos Capa Programas Ejecutables y Conceptos Capa Repositorio de Datos. Los distintos Requerimientos Funcionales (RF) y los Requerimientos No Funcionales (RNF) definidos para cada subdiagnóstico serán las entidades vinculantes entre el Contexto de Infraestructura Tecnológica - CIT, el Diagnóstico Final de Situación de Seguridad - DFSS y los distintos subdiagnósticos. El propósito es que queden representados los conceptos y sus relaciones, en virtud de cómo se plasman en el ámbito de la Seguridad de la Información, la Ingeniería de Requerimientos, y finalmente en el diagnóstico que brindará el SBC.

2) Conocimientos Estratégico

El análisis del conocimiento estratégico permite desarrollar una definición precisa de los cursos de acción modulares que sigue el Experto al desempeñar sus tareas y el flujo de control que gobernará el funcionamiento y el dinamismo del sistema Experto. De esta forma al efectuar la síntesis, en caso de ser necesario, se podrá realizar un reacomodamiento de etapas, pasos, tareas, etc. El conocimiento estratégico se resume a través del Árbol de descomposición funcional, Figura 2. En trazos resaltados en color azul se encuentran los pasos analizados en este primer alcance del prototipo al que se pretende arribar , dejando sin explotar los pasos concernientes

al diagnóstico del nivel de seguridad en la capa de autenticación asumiendo vínculos seguros y procesos de autenticación sólidos y estandarizados.

De acuerdo al trabajo de educción de conocimiento para el diagnóstico y los subdiagnósticos correspondientes se establece el nivel de aceptación que puede presentar una aplicación en materia de Seguridad de la Información. El nivel de aceptación queda expresado como resultado parcial o final ya sea si corresponde al diagnóstico general o a los subdiagnósticos y queda constituido de acuerdo a los niveles de aceptación propuestos en la ERSSI.

Los niveles de aceptación propuestos se describen a continuación, dejando abierta la posibilidad de agregar otros en futuros trabajos, permitiendo que el SBC instaure más diagnósticos. Los niveles propuestos son:

- OPTIMO (no se puede vulnerar por el nivel de protección definido)
- SEGURO (se recomienda monitoreo a fin de detectar vulnerabilidades, fallas y/o ataques a la seguridad)
- INSEGURO (no se garantiza la integridad, disponibilidad y confidencialidad de los datos)

3) Conocimientos Tácticos

Mediante el proceso de adquisición y extracción de conocimiento (fase I), el Experto brinda conocimientos tácticos que especifican cómo el sistema puede utilizar escenarios o hechos conocidos así como hipótesis de los casos presentados a fin de obtener nuevas hipótesis, tanto en situaciones deterministas como en contextos de incertidumbre.

La articulación de los conocimientos tácticos se realiza a través del empleo de seudorreglas que posteriormente se formalizarán a través de reglas en función de la herramienta de desarrollo del prototipo.

Dentro del Modelo Dinámico (o modelo de procesos) se debe definir una jerarquía de tareas, partiendo de la identificación de conocimientos estratégicos. El Experto participa en la realización de este modelo comprobando las metas, submetas, decisiones, acciones, conceptos y atributos que se aplican. Para cada nivel en la jerarquía se definen metas (objetivo), entradas necesarias y salidas producidas.

4) Mapa de conocimiento

El Mapa de Conocimiento (MC) es la síntesis del Modelo Dinámico y del Modelo Estático. Representa la parte estática y dinámica de los conocimientos del Experto. Permite ubicar una relación directa entre el Experto y el Ingeniero en Conocimiento al representar de manera entendible los conocimientos educidos a los usuarios finales. El enfoque a través de MC permite que tales los conocimientos puedan ser empleados e implementados de una forma demostrable, documentable y auditable.

En este contexto el Experto identificó cuatro áreas esenciales para la construcción del MC a fin de arribar al diagnóstico final de situación de seguridad.

Cada subproblema a resolver e instaurando un diagnóstico parcial por cada una de ellas, las áreas son: Nivel de Seguridad en Capa Autenticación, Nivel de Seguridad en Capa Servidor de Aplicaciones, Nivel de Seguridad en Capa Programas Ejecutables y Nivel de Seguridad en Capa Repositorio de Datos, como se exhibe en la Figura 3.

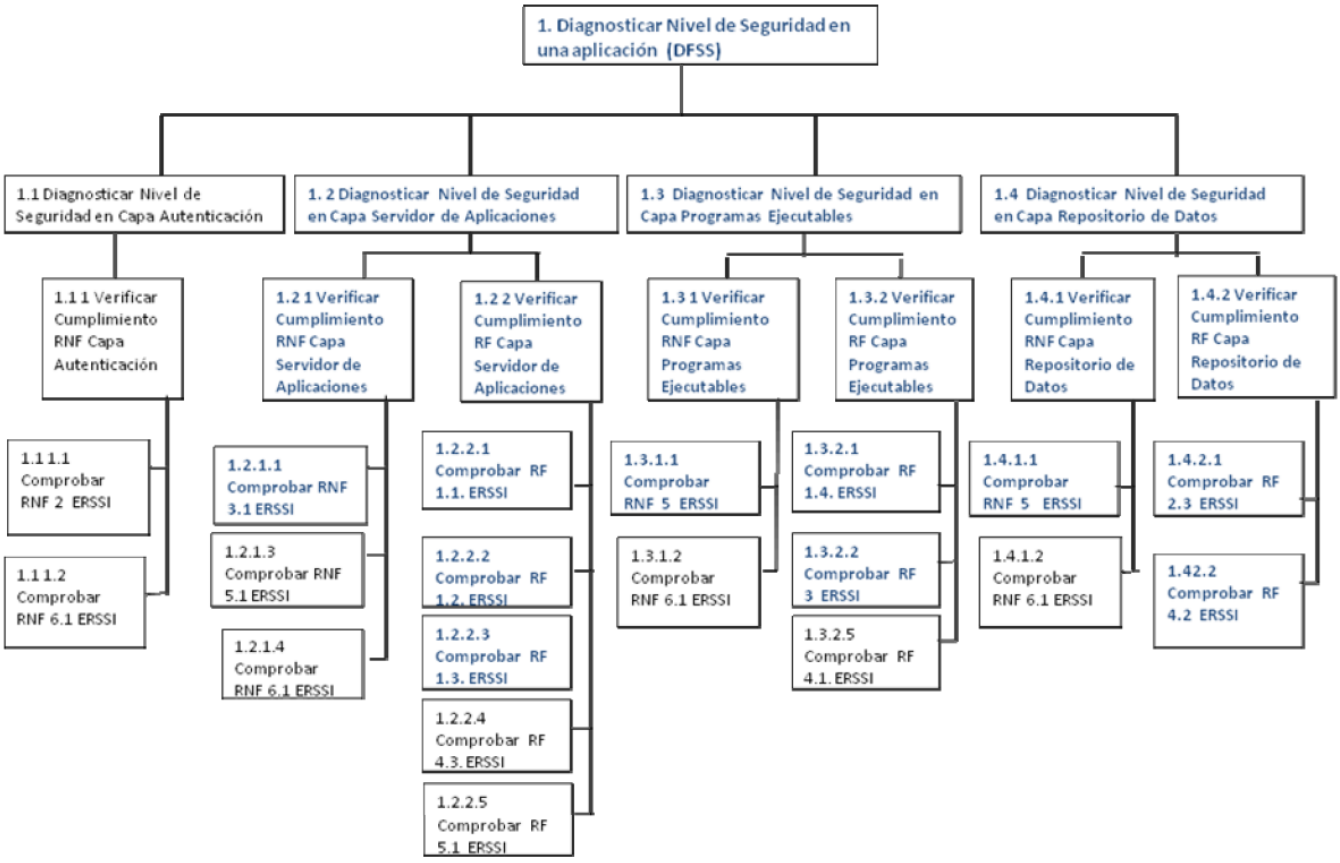


Fig. 2. Árbol de descomposición funcional



Fig. 3. Mapa de conocimiento diagnóstico DFSS

Los MC que se desarrollan a fin de representar el problema de estudio se exhiben a en las siguientes figuras. El Experto identificó tres áreas esenciales para la construcción del MC, facilitando la evaluación de cada subproblema a resolver, las áreas son: Nivel de Seguridad en Capa Servidor de Aplicaciones, Figura 4; Nivel de Seguridad en Capa Programas Ejecutables, Figura 5; Nivel de Seguridad en Capa Repositorio de Datos, Figura 6.

Para finalizar el proceso de conceptualización del conocimiento, el Experto validó el Modelo Estático y el Modelo Dinámico y comprobó el MC a través de distintos juegos de ensayo.

B. Formalización del Conocimiento

La formalización del conocimiento es el resultado obtenido a partir de la conceptualización de conocimientos representada a través del conocimiento fáctico (Tabla Concepto-Atributo-Valor), táctico (seudorreglas) y estratégico (Árbol de descomposición funcional). Establece modelos formales que brindan una representación semi-interna o semi-computable de los conocimientos y conducta del Experto que puedan ser utilizadas por una computadora.

El formalismo de Marcos es una de las técnicas más utilizadas cuando el conocimiento del dominio se organiza en base a conceptos. Los Marcos agregan una tercera dimensión al permitir que los nodos tengan estructuras, que pueden ser valores simples u otros marcos [6] [7]. A través de formalismos de Marcos se representan los conceptos y sus atributos determinados en la fase conceptualización a través del conocimiento fáctico, los conceptos de la tabla concepto-atributo- valor se formalizan en Marcos clase, los atributos del concepto representan las propiedades del Marco. Los valores de cada atributo correspondiente a las propiedades del Marco se detallan a través de las facetas que formulan los valores con los que se puede completar cada propiedad.

1) Marcos Clase y Marcos Instancia

Los Marcos Clase se utilizan para representar conceptos de la tabla Concepto-atributo-valor así como situaciones genéricas proporcionadas por un conjunto de características, unas con valores determinados y otras sin valores asignados que son comunes al concepto. Los Marcos Clase representados son: DIAGNIVELSEG (diagnóstico general nivel de seguridad), DIAGCSAP (diagnóstico capa servidor de aplicaciones), DIAGCPE (diagnóstico capa programas ejecutables), DIAGCRD (diagnóstico capa repositorio de datos), Entorno de Testeo, Entorno de Producción, Gestión de Liberaciones, Separación de ambientes, Control de Programas Ejecutables, Vulnerabilidades de las aplicaciones Web, Control de Programas fuente, Resguardo de Programas fuente, Recupero de Programas fuente, Capa de Seguridad, Resguardo de Datos, Recupero de Datos, Acceso a Datos.

Los Marcos Instanciados se utilizan para representar conceptos particulares al momento de efectuar la tarea de diagnóstico, es decir cuando se está evaluando un escenario particular.

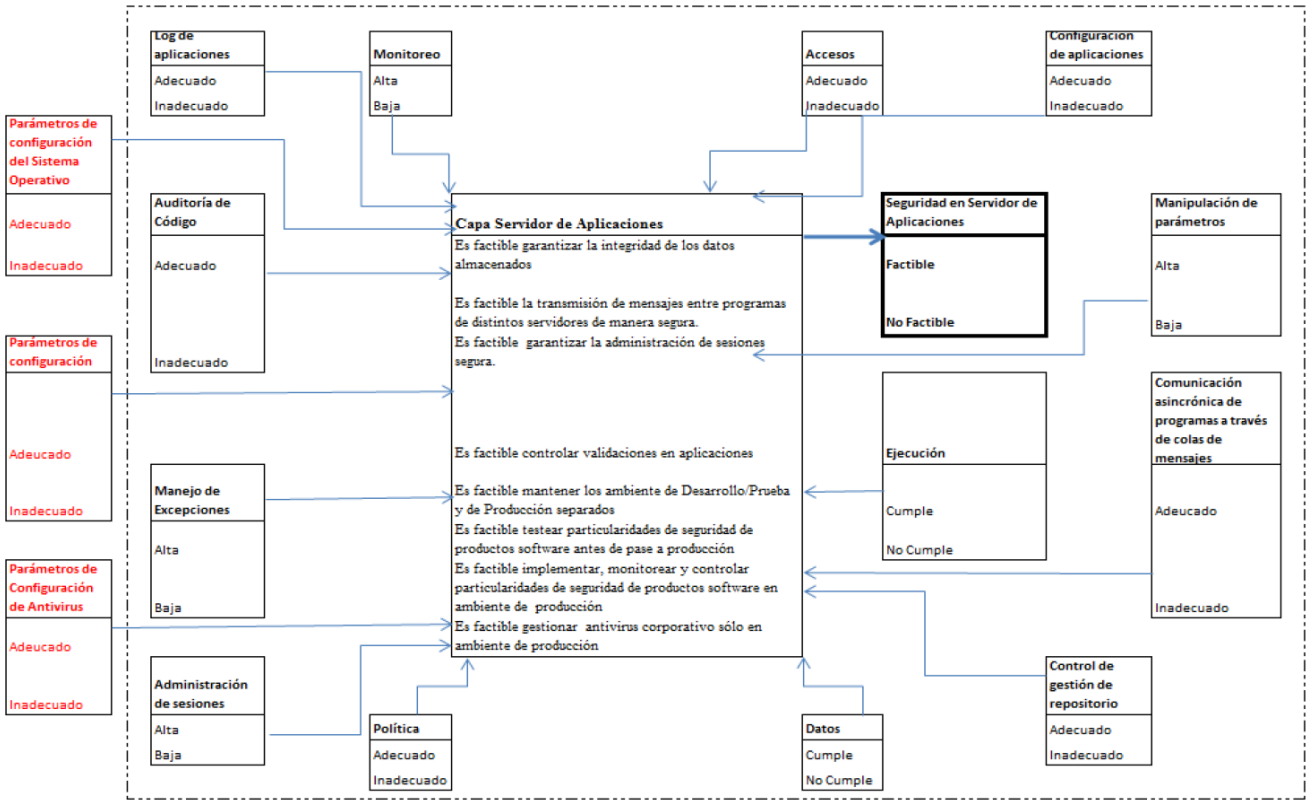


Fig. 4. Nivel de Seguridad en Capa Servidor de Aplicaciones

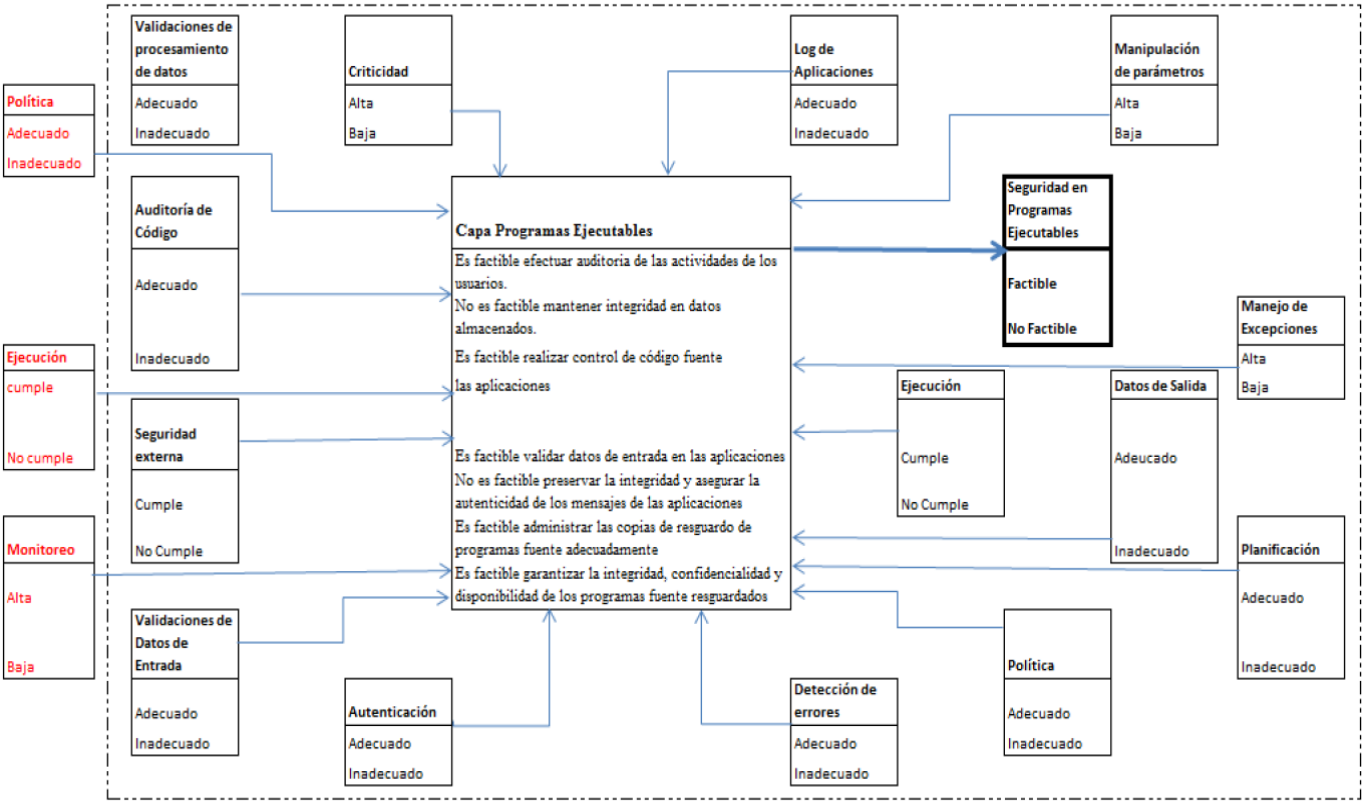


Fig. 5. Nivel de Seguridad en Capa Programas Ejecutables

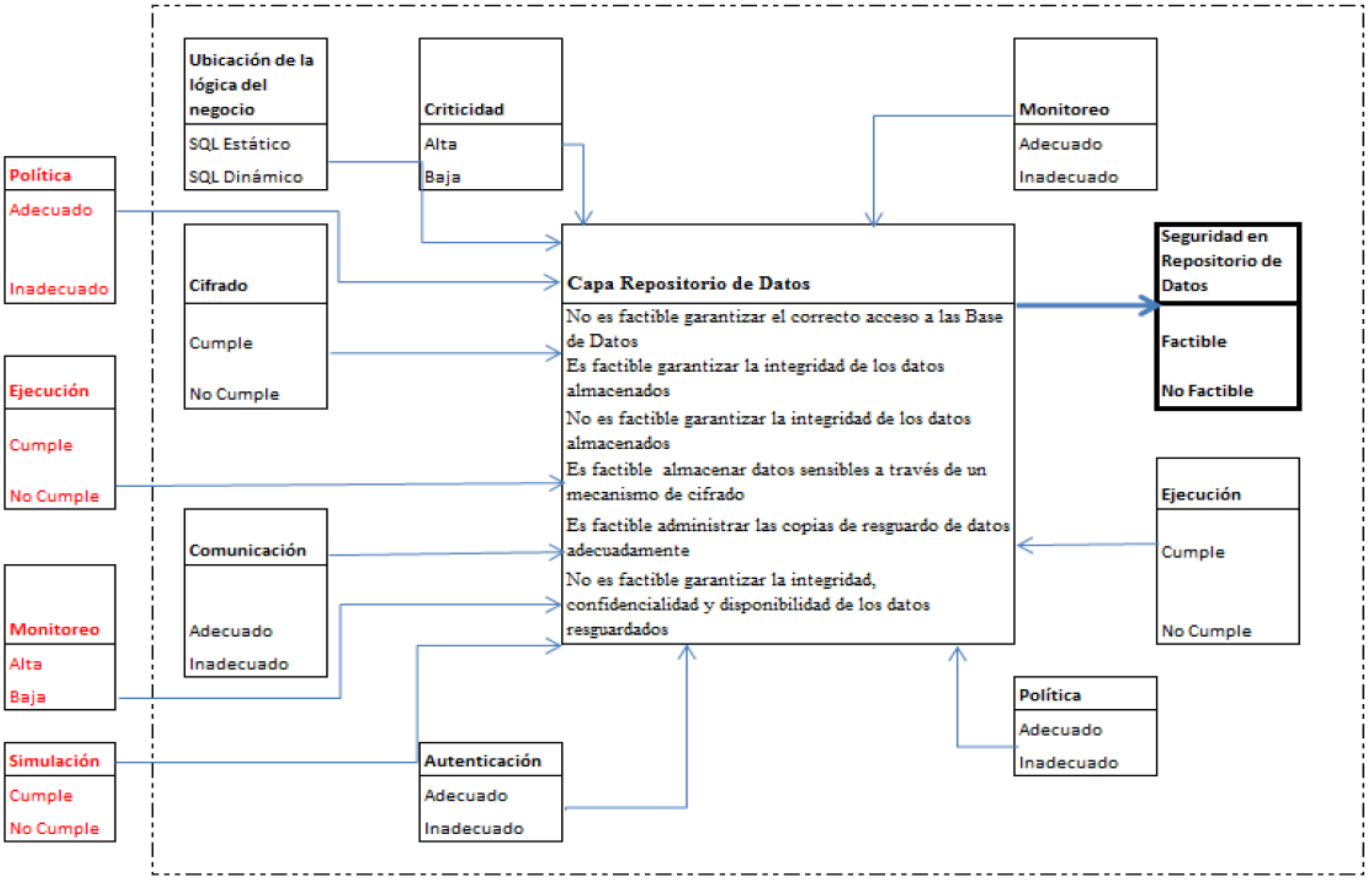


Fig. 6. Nivel de Seguridad en Capa Repositorio de Datos

Los Marcos instanciados representados son: DIAGNIVELSEG Presente (diagnóstico general nivel de seguridad), DIAGCSAP Presente (diagnóstico capa servidor de aplicaciones), DIAGCPE Presente (diagnóstico capa programas ejecutables), DIAGCRD Presente (diagnóstico capa repositorio de datos), Entorno de Testeo Presente, Entorno de Producción Presente, Gestión de Liberaciones Presente, Separación de ambientes Presente, Control de Programas Ejecutables Presente, Vulnerabilidades de las aplicaciones Web Presente, Control de Programas fuente Presente, Resguardo de Programas fuente Presente, Recupero de Programas fuente Presente, Capa de Seguridad Presente, Resguardo de Datos Presente, Recupero de Datos Presente, Acceso a Datos Presente. En esta ocasión los Marcos Instanciados fueron utilizados en el momento de implementación y explotados para los casos de prueba. Esto considerando que el Experto, a través de las entrevistas, brindó los valores por defecto para completar dichos marcos, así como valores para conformar los casos basales para efectuar las pruebas y validar el resultado arrojado por el sistema.

2) Relaciones entre conceptos

El formalismo de marcos permite representar las relaciones del dominio, con relaciones entre marcos clase, entre marcos instancias y entre marcos clase y marcos instancias, estableciendo de esta manera un sistema basado en marcos (SBM). El significado de las relaciones es el siguiente:

- La relación "Se basa en": representa el diagnóstico parcial de cada uno de los dominios a evaluar y que contribuirá a obtener el Diagnostico Final de Situación de Seguridad (DFSS) de una aplicación.
- La relación "Se comprueba": representa el nivel de cumplimiento de cada uno de los requerimientos funcionales (RF) y los requerimientos no funcionales (RNF) desarrollados en la ERSSI.
- La relación "Considera el": representa el peso que tiene el nivel de cumplimento de aquellos RF y RNF que son considerados "de soporte" para la evaluación del DIAGCSAP, dentro del alcance planteado. Para este caso los valores usados son valores por defecto o aquellos valores indicados por el Experto en los casos de prueba.

C. Implementación del Sistema Experto

Como herramienta para desarrollo se utilizó Kappa-PC que es una herramienta que facilita la implementación de sistemas que hayan sido formalizados en base a marcos. Brinda un entorno de desarrollo que facilita la construcción rápida, permitiendo un ciclo de vida en donde en cada iteración se incrementen los conocimientos y así lograr un desarrollo basado en prototipado incremental congruente con las bases propuestas en la Metodología IDEAL [5] [6] [7] [11] [12]. Para la implementación del Sistema Experto se realizaron los pasos que se detallan a continuación:

1- Declaración de la base de conocimientos formalizada en Marcos, a través de la herramienta para la representación de Marcos Clase y Marcos Instancias. Se declararon los objetos clase correspondientes a: diagnóstico general nivel de seguridad, diagnóstico capa servidor de aplicaciones, diagnóstico capa programas ejecutables, diagnóstico capa repositorio de datos, Entorno de Testeo, Entorno de Producción, Gestión de Liberaciones, Separación de ambientes, Control de Programas Ejecutables, Vulnerabilidades de las aplicaciones Web, Control de Programas fuente, Resguardo de Programas fuente, Recupero de Programas fuente, Capa de Seguridad, Resguardo de Datos, Recupero de Datos y Acceso a Datos.

2- Definición de las propiedades de clase de los diferentes marcos, utilizando los slots que se pueden definir en cada objeto. Para cada slot se define cardinalidad, valor tipo y valor permitido.

3- Incorporación de reglas para cada una de las áreas que forman el dominio del Problema hasta llegar a las correspondientes reglas del Diagnóstico General de Seguridad, en concordancia con las seudorreglas construidas en el marco de la Conceptualización del Conocimiento.

4- Correspondencia del sistema con la estructura de razonamiento de encadenamiento hacia atrás. Se desarrollan los objetivos de acuerdo al siguiente orden:

- Subdiagnóstico Capa Servidor de Aplicaciones.
- Subdiagnóstico Capa Programas Ejecutables.
- Subdiagnóstico Capa Repositorio de Datos.
- Diagnóstico General de Seguridad.

5- Desarrollo de pantallas gráficas correspondientes a menús de ingreso/selección de datos por parte del usuario así como visualización de resultados correspondiente a los distintos subdiagnósticos.

6- Adecuación de las distintas interfaces, conforme a las sugerencias del usuario. La forma de navegar el sistema se muestra en la Figura 7.

7- Realización de diversas sesiones de pruebas con el Experto a fin de evaluar la facilidad de navegación de las Interfaces de usuario. A su vez se efectuaron pruebas funcionales del sistema experto en lo relacionado a la base de reglas, dando un resultado ampliamente satisfactorio en relación a los requerimientos del usuario.

1) Mapa de pantallas

A continuación se exhibe la manera en que el usuario puede navegar por las distintas pantallas que tiene el sistema.

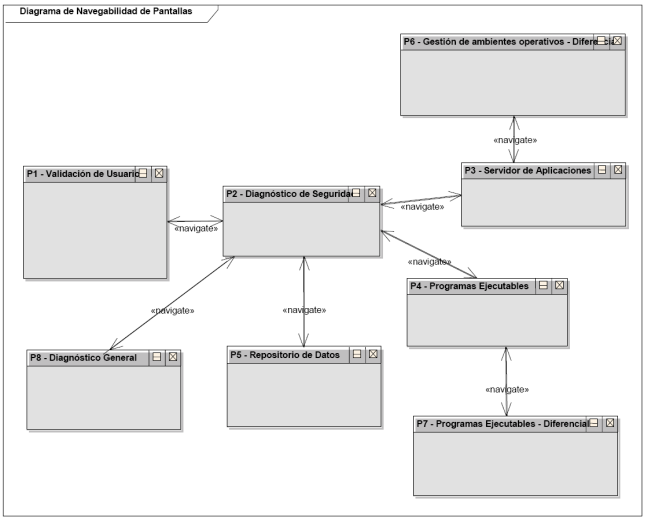


Fig. 7. Navegabilidad de interfaces de usuario

2) Interfaces de Usuario

A modo ilustrativo se exhiben algunas pantallas que forman parte del prototipo construido con la herramienta Kappa-PC.

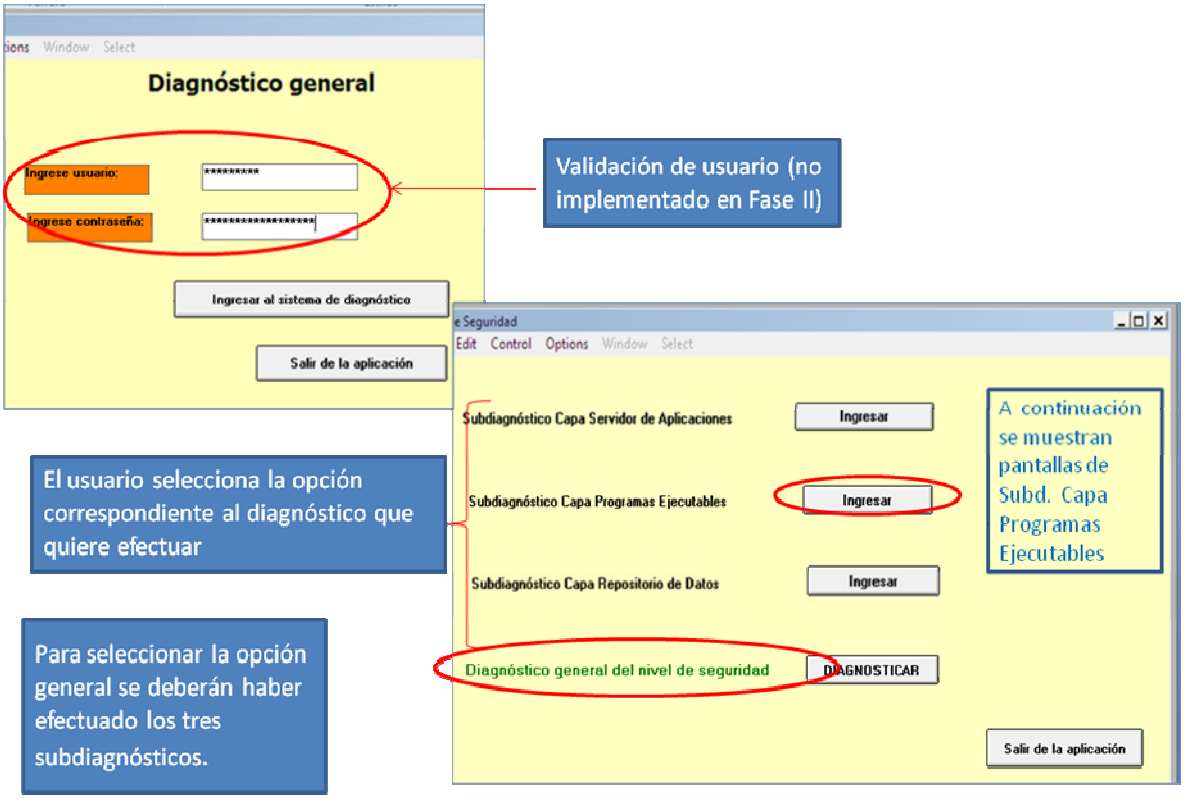


Fig. 8. Interfaces: Ingreso y validación de Usuario (a) – Menú de selección de subdiagnósticos (b)

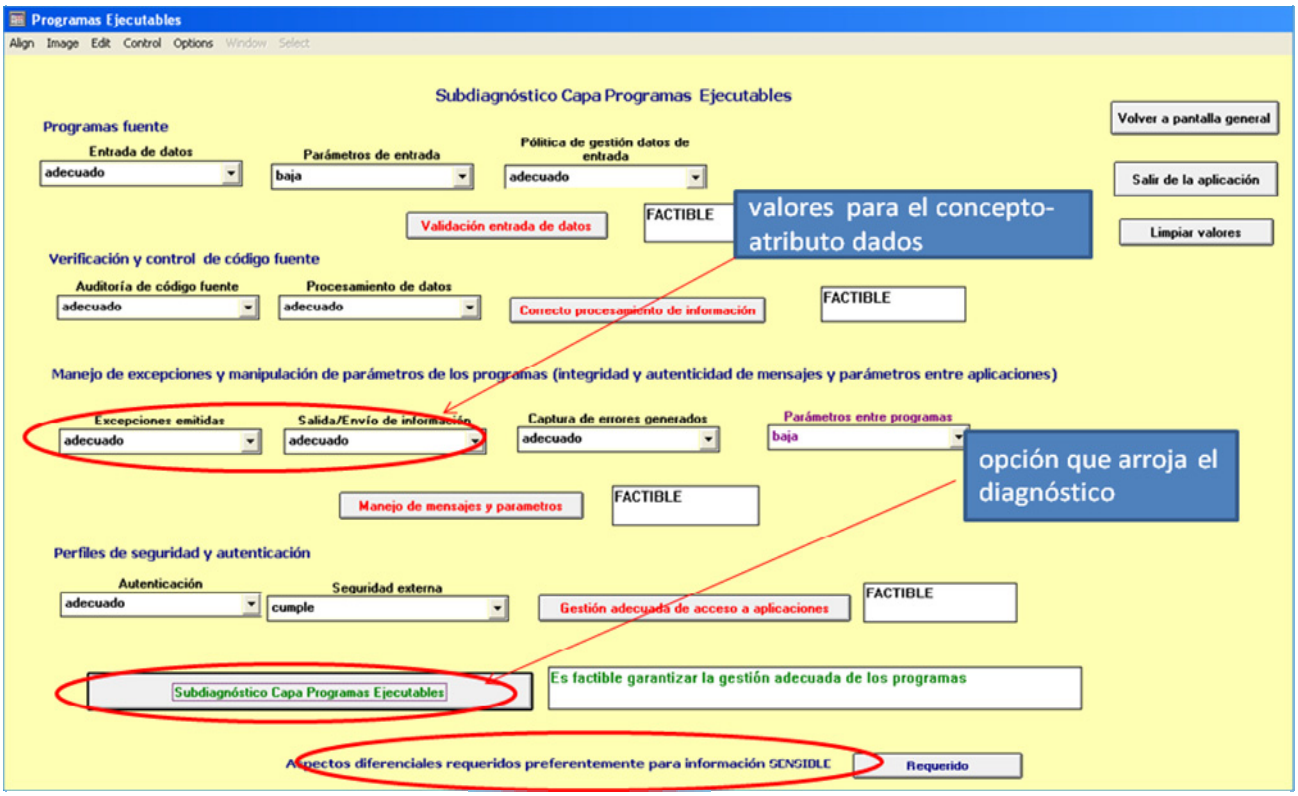


Fig. 9. Interfaz: Subdiagnóstico de Programas Ejecutables

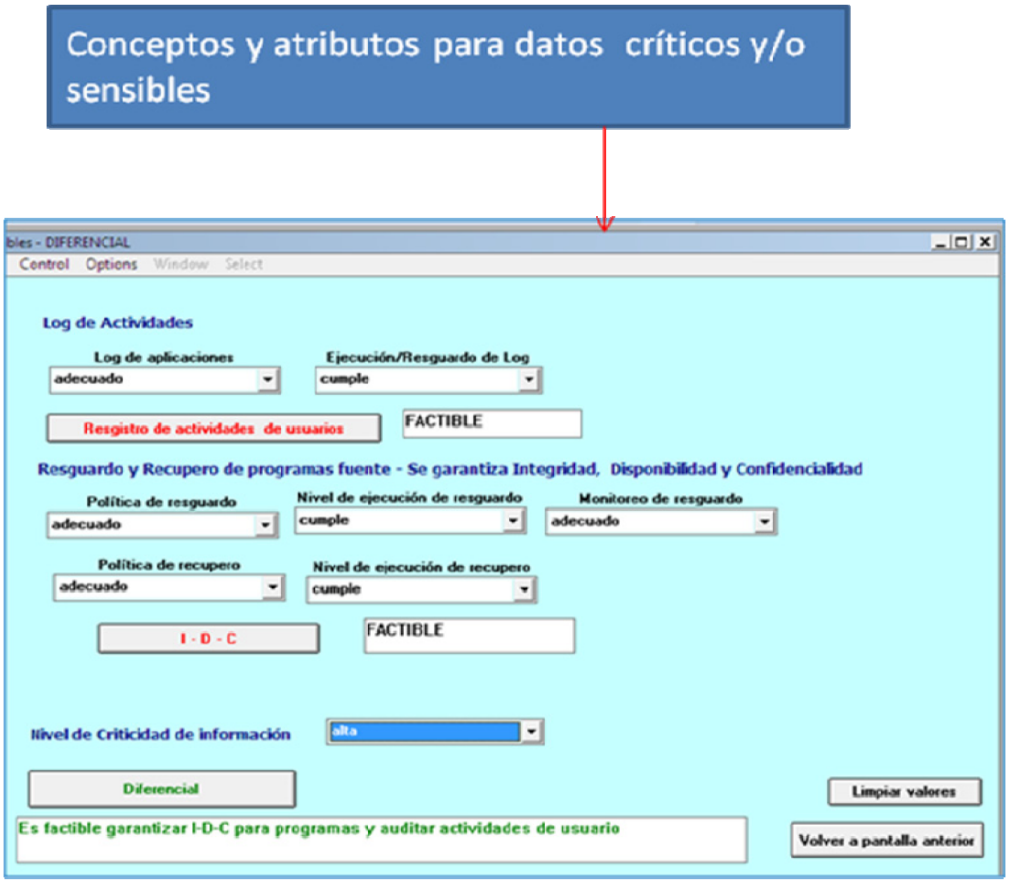


Fig. 10. Interfaz: Subdiagnóstico de Programas Ejecutables - Diferencial

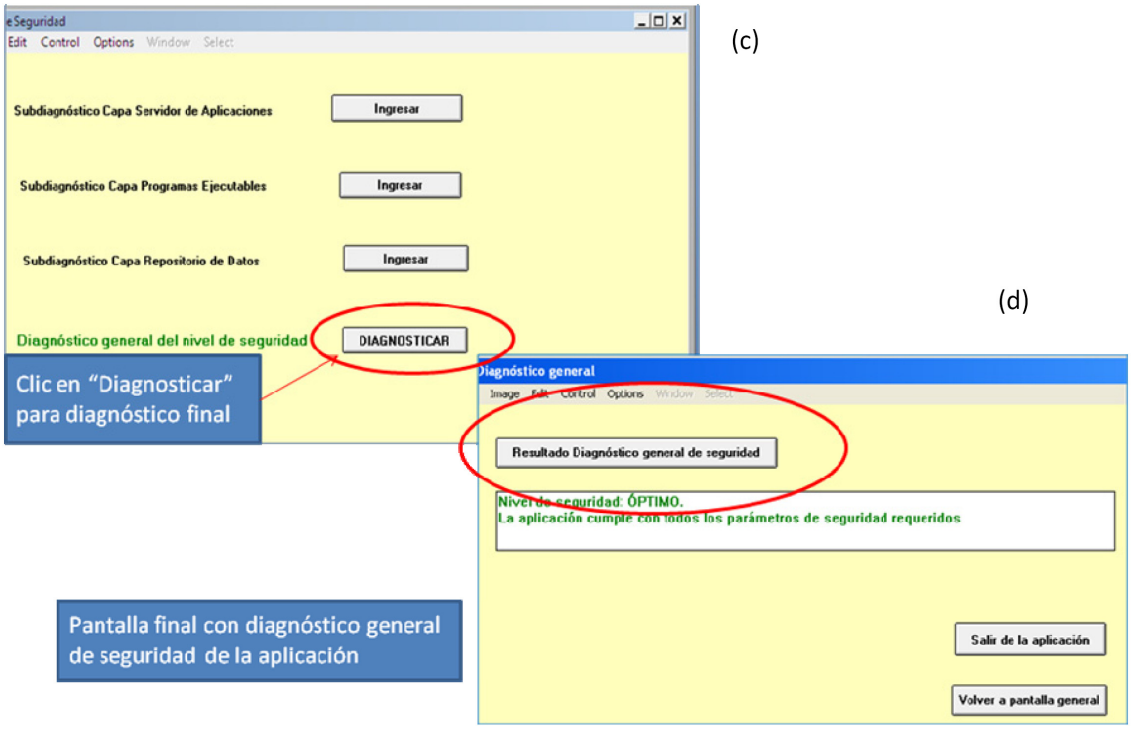


Fig. 11. Interfaces Diagnóstico Final (c) - Resultado obtenido de la evaluación (d)

3) Resumen de casos de prueba para validar el modelo propuesto

La siguiente tabla condensa la prueba funcional que efectuó el Experto. Es el resultado de diez (10) casos de prueba, instancias de casos base, que se aplicaron para la evaluación funcional del Sistema. Los mismos fueron presentados y evaluados por el experto y se basan en sus actividades cotidianas sobre su conocimiento basal.

TABLA I. RESUMEN DE PRUEBAS FUNCIONALES

Caso de prueba	Resultado Obtenido (Diagnóstico del Sistema)	Resultado Esperado (Diagnóstico del Experto)
1 - Gestión de Turnos	Nivel de seguridad: ÓPTIMO La aplicación cumple con todos los parámetros de seguridad requeridos	OPTIMO. La aplicación pasa satisfactoriamente requerimientos mandatorios y opcionales de seguridad.
2 - FTP Seguro	Nivel de seguridad: ÓPTIMO La aplicación cumple con todos los parámetros de seguridad requeridos	OPTIMO. La aplicación pasa satisfactoriamente requerimientos mandatorios y opcionales de seguridad.
3 - SATCS – SAT Control Stage	Nivel de seguridad: ÓPTIMO La aplicación cumple con todos los parámetros de seguridad requeridos	OPTIMO. La aplicación pasa satisfactoriamente requerimientos mandatorios y opcionales de seguridad.
4 - Gestión de Accesos automáticos	Nivel de seguridad: SEGURO. La aplicación cumple con los parámetros de seguridad. No se garantiza gestión adecuada de datos sensibles.	SEGURO. La aplicación pasa los requerimientos mandatorios de seguridad. No verifica adecuación de ambiente de desarrollo/prueba.
5 - Sistema Integral de Delegaciones	Nivel de seguridad: SEGURO. La aplicación cumple con los parámetros de seguridad. No se garantiza gestión adecuada de datos sensibles.	SEGURO. La aplicación pasa los requerimientos mandatorios de seguridad. No verifica niveles apropiados de IDC ni datos sensibles.
6 - Emulador Web	Nivel de seguridad: SEGURO. La aplicación cumple con los parámetros de seguridad. No se garantiza gestión adecuada de datos sensibles.	SEGURO. La aplicación pasa los requerimientos mandatorios de seguridad. No verifica niveles apropiados de IDC.
7 - Base Unificada de Administración de Prestaciones	Nivel de seguridad: SEGURO. La aplicación cumple con los parámetros de seguridad. No se garantiza gestión adecuada de datos sensibles.	SEGURO. La aplicación pasa los requerimientos mandatorios de seguridad. No verifica niveles apropiados de IDC ni datos sensibles.
8 - Gestión de usuarios por Web	Nivel de seguridad: INSEGURO. Recomendación: volver a evaluar la aplicación, los datos y el entorno.	INSEGURO. La aplicación no pasa los requerimientos mandatorios y diferenciales de seguridad.
9 - Cambio de Delegación por e-mail	Nivel de seguridad: INSEGURO. Recomendación: volver a evaluar la aplicación, los datos y el entorno.	INSEGURO. La aplicación no pasa los requerimientos mandatorios y diferenciales de seguridad.
10 - Compra Electrónica Web	Nivel de seguridad: INSEGURO. Recomendación: volver a evaluar la aplicación, los datos y el entorno.	INSEGURO. La aplicación no pasa los requerimientos mandatorios y diferenciales de seguridad.

III. CONCLUSIONES

Se detallan los aportes que el presente trabajo ofrece a la problemática específica de la seguridad de las aplicaciones de gestión, teniendo en cuenta los siguientes aspectos:

- Propone un modelo de un Sistema Basado en Conocimiento (SBC), capaz de dar respuesta al análisis de los niveles de seguridad de aplicaciones de gestión.
- Sistematiza y documenta, con metodología de Sistemas Expertos, el conocimiento requerido para el área de la seguridad de aplicaciones de gestión.
- Fija las bases para la realización de un Sistema Experto que asiste en el análisis y evaluación de Especificación de Requisitos de Software (ERS), que incorpora como

propuesta los aspectos de seguridad, desde el punto de vista de la Ingeniería de Requerimientos (IR).

- Aplica, para el área de Ingeniería en Conocimiento, un marco metodológico a través de la metodología IDEAL, asegurando el desarrollo y posterior crecimiento del Sistema Experto, en los aspectos relativos al mantenimiento del conocimiento.
- Documenta y modela la educción y extracción de conocimiento. Se apoya en técnicas de adquisición de conocimiento, la elaboración de una taxonomía de los requisitos funcionales y no funcionales, la conceptualización de los conocimientos estratégicos, facticos y tácticos para el domino de seguridad de las aplicaciones, la formalización y la posterior implementación de un prototipo del SBC, validado a través de casos de pruebas determinados por el experto en Seguridad de la Información.
- Sostiene la aplicación de la solución a través de un SBC, sobre la base de un Test de Viabilidad que permite, desde una etapa temprana, establecer un umbral de éxito que vale como incentivo para continuar con el desarrollo del Sistema Experto.

IV. Trabajo Futuro

Se exponen las futuras líneas de investigación que se pueden tomar en cuenta con el objetivo de continuar con el presente trabajo incrementando las funcionalidades del Sistema Experto y ampliando el conocimiento del sistema a través de la incorporación de módulos para el manejo de situaciones específicas. Del trabajo efectuado, así como de la experiencia adquirida, surgen las siguientes propuestas:

- Ampliar el modelo de conocimientos del SBC optimizando la taxonomía de requerimientos funcionales y no funcionales con la incorporación de temas vinculados con la gestión de activos, la seguridad del personal, la seguridad física y ambiental, la gestión de la comunicación y las operaciones ente otros aspectos de la Seguridad de la Información.
- Ampliar el alcance del SBC incorporando conceptos relacionados con la seguridad en la Capa Autenticación, ampliando conceptos relacionados con la seguridad de la Capa Servidor Aplicaciones, Capa Programas Ejecutables y Capa Repositorio de Datos, así como otros que puedan agregar valor a partir de la explotación de la taxonomía de requerimientos funcionales y no funcionales.
- Investigar sobre herramientas de soporte para la gestión de datos que puedan incorporarse al sistema incrementando de esta manera su robustez y permitiendo efectuar trazabilidad de los resultados.
- Extender las funcionalidades de explotación del SBC por parte de los usuarios y expertos remotos a través de una capa de interfaz de usuario vía WEB.
- Investigar la viabilidad de la aplicación de reglas difusas en el domino de estudio.

Agradecimientos

A la Escuela de Posgrado de la Universidad Tecnológica Nacional y al Instituto de Sistemas Inteligentes de la Universidad de Morón. Por último, y muy especialmente, al Experto en Seguridad de la Información por todo el trabajo de campo realizado, por sus grandes aportes al trabajo y por el tiempo brindado desinteresadamente.

Referencias bibliográficas

[1] ArCERT (Coordinación de Emergencias en Redes Teleinformáticas de la Administración Pública), Subsecretaría de Gestión Pública) www.arcert.gov.ar.

[2] Bajarlía, M.V. 2010."Modelo del conocimiento en Seguridad de aplicaciones".Tesis de Magister en Ingeniería en Sistemas de Información, UTN, Escuela de Posgrados, publicado.

[3] Bajarlía, M.V., Ierache, J., Eterovic, J. 2010. "Modelo de Sistema Basado en Conocimiento para el Análisis de la Seguridad de la Información en el Contexto de los Sistemas de Gestión". XVI Congreso argentino de ciencias de la computación- V Workshop Arquitectura, Redes y Sistemas Operativos (WARSO), CACIC 2010. Universidad de Moron, 18 al 22 de Octubre, 2010. ISBN 78-950-9474-49-9, publicado.

[4] Bajarlía, M.V., Ierache, J., Eterovic, J. 2008. "Elaboración de Especificación de Requerimientos de Seguridad en el desarrollo de Sistemas de Información basado en la Modelización de Conocimientos". X Workshop de Investigadores en Ciencias de la Computación - WICC 2008, Universidad Nacional de La Pampa, 5 y 6 de Mayo, 2008. ISBN 978-950-863-101-5, publicado.

[5] Fernández Galán, S., González Boticario, J., Mira Mira, J.1998. Problemas resueltos de Inteligencia Artificial Aplicada. Búsqueda y representación. Addison-Wesley.

[6] García Martinez R., Britos P.2004. Ingeniería de Sistemas Expertos. Nueva Librería.

[7] Giarratano, J., Riley, G.2000. Sistemas Expertos Principios y Programación. Thomson International.

[8] Harrison, R. 1999. ASP/MTS/ADSI Web Security. Longman.

[9] HISPASEC SISTEMAS. www.hispasec.com .

[10] IEEE (Institute of Electrical and Electronics Engineers), www.ieee.org.

[11] Intellicorp. 1992. Kappa PC Quick Start. Intellicorp Inc.

[12] Intellicorp. 1992. Kappa PC User Guide Intellicorp Inc.

[13] ISECOM (Institute for security and open methodologies) www.isecom.org.

[14] ISO (International Organization for Standardization), www.iso.org.

[15] ISO/IEC 27001, 2006. Gestión de la Seguridad de la Información.

[16] Jaworski, J., Perrone, P.J. 2000. Seguridad en Java. Prentice Hall.

[17] Kaeo, M. 2000. Diseño de Seguridad en Redes. Pearson Educación.

[18] Maiwald, E.2005. Fundamentos de la seguridad de redes. Conocimientos esenciales a tu alcance. McGraw-Hill.

[19] Maté Hernández, J.L., Pazos Sierra J. 1988. Ingeniería del Conocimiento. Diseño y construcción de sistemas expertos. Sepa S.A.

[20] MinsKy M. 1975. A framework for representing Knowledge.McGraw Hill.

[21] NIST (National Institute of Standards and Technology, Technology Administration U.S. Department of Commerce), www.nist.gov .

[22] Piattini Velthuis, M., Del Peso Navarro, E.2001. Auditoría informática un enfoque práctico. Alfaomega Grupo Editor Argentino S.A.

[23] PKI (Public Key Infraestructure), Subsecretaría de Gestión Pública www.pki.gov.ar

[24] Pressman R.2006. Ingeniería del Software, un enfoque práctico. McGraw Hill.

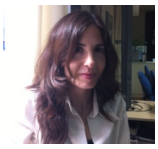
[25] Rusell, S.J., Norvig, P. 2004. Inteligencia Artificial. Pearson Educación.

[26] Seguridad en Java www.java.sun.com/products/jaas.

[27] Seguridad en Windows www.microsoft.com/security

[28] Sommerville, I. 2002. Ingeniería de Software. Addison Wesley.

[29] Stallings, W. 2004. Fundamentos de la seguridad en redes. Aplicaciones y estándares. Pearson Educación.



María Victoria Bajarlía Magister en Ingeniería en Sistemas de Información, Universidad Tecnológica Nacional en 2010. Especialista en Ingeniaría en Sistemas de Información, Universidad Tecnológica Nacional en 2009. Actualmente es Analista de Calidad en Proyectos de T.I. en el sector público. Docente adjunta en Universidad de Ciencias Empresariales y So-ciales en el área de Programación, docente auxiliar en Universidad Tecnológica Nacional en el área de Sistemas. En el campo de la educación cuenta con intereses en po-sicionamiento en actividades de I+D, gestión académica de grado y pos grado, transferencia de tecnología, publi-caciones y artículos técnicos, proyectos de investigación, etc. En el ámbito laboral la expectativa es continuar con el liderazgo funcional, en el marco de la gestión de la calidad aplicada a proyectos de TI.



Jorge Eterovic. Magister en Dirección de Sistemas de Información por la Universidad del Salvador, Especialista en Criptografía y Seguridad Teleinformática por la Escuela Superior Técnica - Universidad del Ejército. Director de la carrera de Ingeniería en Informática y Profesor titular de la materia Auditoria y Seguridad Informática en la Universidad de Morón. Profesor Asociado de las materias Proyecto y Auditoria y Seguridad Informática en la Universidad Nacional de La Matanza. Docente de posgrado en las Universidades Austral, del Salvador y Nacional de La Matanza. Categorizado como Investigador Principal en el régimen del CONICET.



Jorge Ierache Doctor en Ciencias Informáticas por la Universidad Nacional de la Plata, Magíster en Ingeniería del Conocimiento por la Universidad Politécnica de Madrid. Profesor Adjunto regular del área Ingeniera de Software Facultad de Ingeniería Universidad Nacional de Buenos Aires, Director del Laboratorio de Sistemas Avanzados de Información de FIUBA y del Instituto de Sistemas Inteligentes y Enseñanza Experimental de la Robótica (ISIER) de la Universidad de Morón.

Gestión y Desarrollo de Proyectos de Software: Metodología Ágil basada en Telecomunicaciones MATe

Marisa Cecilia Tumino[1], Juan Manuel Bournissen[1], Claudio Bracalenti[2], Eric Schlemper[3], Silvio Kucharski[1]
1. Instituto de Ingeniería del Software. Universidad Adventista del Plata. Libertador San Martín, Entre Ríos. Argentina
2. Universidad Tecnológica Nacional Facultad Regional Santa Fe, Argentina
3. Sanatorio Adventista del Plata. Libertador San Martín, Entre Ríos Argentina
I3@uap.edu.ar

***Abstract*—La Metodología Ágil basada en Telecomunica-ciones (MATe) es una propuesta metodológica orientada al progreso de emprendimientos informáticos que permite el desarrollo de software por parte de grupos pequeños con escasos recursos. La metodología contempla el trabajo semi-sincrónico, utilizando los adelantos, y las reducciones de costos asociadas en materia de comunicaciones y de recursos tecnológicos, contribuyendo a la fluidez y a la simplicidad del trabajo en un ambiente totalmente virtualizado y libre de gran parte de los costos fijos comunes a una Software factory convencional. Se han rescatado aspectos exitosos de otras metodologías tales como la reunión diaria de Scrum y la programación en parejas de XP, aunque las mencionadas reuniones se desarrollen en salas virtuales y las parejas de programadores residan en lugares distantes.**

***Palabras claves* —Metodología ágil, Teletrabajo, desarrollo semi-sincrónico, Trabajo Freelance.**

I. Introducción

La filosofía de las metodologías ágiles otorgan mayor valor al individuo, a la colaboración con el cliente y al desarrollo incremental del software con iteraciones muy cortas, mostrando su efectividad en proyectos con requisitos cambiantes o cuando se exige reducir los tiempos de desarrollo, manteniendo la calidad [1].

El presente trabajo tiene como objetivo primario proponer una metodología ágil que facilite el desarrollo de aplicaciones para dispositivos móviles, utilizando, como plataforma de desarrollo, medios que permitan el trabajo a distancia y puedan incluso soportar desplazamientos de los involucrados. El objetivo secundario es el desarrollo de un producto que facilite la gestión de los proyectos, utilizando la metodología propuesta.

II. Demandas del emprendimiento

En el presente, aun más que en el pasado, fundar una empresa puede llegar a transformarse en una actividad que requiera gran inversión de capital. La fiebre de los móviles con pantalla táctil [2] ha revolucionado el mercado de todo buen negocio de las aplicaciones móviles. Y es que estos terminales se han convertido en el ordenador del futuro con el valor añadido de disponibilidad total y de la comodidad y portabilidad.

Los emprendedores recién egresados de la universidad enfrentan dos grandes problemas: (a) las distancias que median entre las residencias de los miembros de equipo de trabajo y (b) la demanda de capital para fundar una empresa de sede tangible.

Las metodologías más usuales, y desde donde se han adoptado los principios básicos del desarrollo de aplicaciones para dispositivos móviles, tales como SCRUM y eXtreme Programing (XP), sufren limitaciones condicionadas a las localizaciones físicas de los miembros del stakeholders, considerados éstos como quienes están afectados por las actividades de una empresa. Por su parte, para efectuar un desarrollo de software efectivo se requiere un ambiente acondicionado con los equipos tecnológicos y los medios de telecomunicaciones apropiados, por lo que se debe invertir además en los impuestos y servicios que demanda este tipo de emprendimiento.

Sintetizando los recursos necesarios para la puesta en marcha de un emprendimiento de desarrollo de software son los siguientes: (a) recursos humanos, (b) ámbito de trabajo, (c) hardware de desarrollo y comunicaciones, (d) software, (e) capital, (f) tiempo y (g) una metodología apta para el trabajo distribuido.

De la lista precedente resulta evidente que en principio el único recurso abundante con el que puede llegar a contar un recién graduado es el tiempo.

En virtud de las condiciones demandadas por las nuevas modalidades de desarrollo Freelance, donde los miembros de los equipos se encuentran ubicados en diferentes puntos geográficos, se considera necesario elaborar una propuesta idónea que dé respuestas a estas limitaciones y a las expectativas del campo de desarrollo de sistemas. Para ello surgen replanteos para cada uno de los recursos enumerados.

A. Recursos humanos

La mecánica de trabajo se basa en la existencia de un grupo inicial de pares, es decir de profesionales que en un sentido técnico gozan de similares conocimientos y entre quienes no se encuentran razones para proponer un orden jerárquico. La organización de los involucrados debe darse en forma espontánea, asumiendo cada uno un rol consensuado y basado en la confianza recíproca. Los roles a desempeñar no implican una escala de poder sino, y esto es de capital importancia, una forma de obtener lo mejor de cada uno de los integrantes. Conforme prospere el emprendimiento es lógico que incorpore nuevo personal, pudiéndose establecer en este caso una jerarquía o continuar con el esquema organizativo inicial. En todo emprendimiento debe haber gestión, producción y

servicios de apoyo. La idea es dividir las necesidades en roles y asignar en forma consensuada uno o varios de ellos a cada miembro del equipo.

B. Ámbito de trabajo

Disponer de un lugar común puede resultar más difícil de lo que parece por dos motivos:

- Alquilar o comprar un inmueble comercial requiere comúnmente una gran inversión de capital.
- Coincidir físicamente en una ciudad implica para algunos mudarse, con los consecuentes costos. La idea es entonces virtualizar el ámbito de trabajo, permitiendo a los emprendedores ejecutar sus tareas desde lugares remotos y compartir un universo virtual en el que se desarrollen sus labores.

C. Hardware de desarrollo y comunicaciones

Lejos están los días en que un equipo de computación era asequible sólo para unos pocos. Hoy, por el contrario, en la mayoría de los hogares de alumnos o graduados universitarios las PCs, notebooks, módems y routers simplemente son parte del equipamiento estándar. La ventaja de trabajar en un ambiente distribuido y virtualizado es, justamente, la de poder contar con esta infraestructura preexistente. Sin embargo para esta propuesta también es imprescindible contar con un smartphone o una tableta que corra Android para el correcto testeo de los productos desarrollados, más allá de los emuladores que se encuentran disponibles en otras plataformas como Windows, Mac o las distintas distribuciones Linux. La elección de Android se sustenta en que es la plataforma móvil más extendida del mercado. Muchas de las posibilidades las brinda Android [3].

D. Software

El interés por la calidad en el desarrollo de software crece de forma continua a medida que los clientes se vuelven más selectivos y comienzan a rechazar productos poco fiables o que realmente no dan respuesta a sus necesidades [4]. Si bien la metodología propuesta podría utilizarse en el desarrollo de cualquier tipo de software, y para cualquier plataforma, este trabajo se ha circunscripto inicialmente a la producción para dispositivos móviles y específicamente para aquellos que soportan Android. Se ha proyectado la propuesta sobre la premisa de trabajar preferentemente con productos gratuitos, atendiendo a las razonables necesidades de los destinatarios de esta metodología. En muchos casos implica la aceptación de publicidad pero que representa un costo razonable de la gratuidad.

E. Capital

Las necesidades de capital inicial son mínimas. Las erogaciones necesarias no dejan de ser las mismas que las cotidianas, más allá de la existencia de un emprendimiento productivo. Como ejemplo vale decir que el grupo de investigación pudo trabajar con equipamiento propio al que solo se le debió sumar auriculares con micrófonos y una tableta genérica de muy bajo costo.

F. Tiempo

El tiempo es un recurso que se asume como abundante, considerando que los involucrados están convencidos de las bondades de trabajar para sí mismos. No obstante su disponibilidad, este recurso debe manejarse con criterio pues la ausencia de una cabeza visible que imponga metas puede conducir a una relajación que conspire contra el alcance del objetivo. Deben fijarse políticas de uso del tiempo, tanto del compartido como del individual, y pautas para verificar el cumplimiento de dichas políticas.

III. La metodología MATe

En virtud de las condiciones demandadas por las nuevas modalidades de desarrollo Freelance, donde los miembros de los equipos se encuentran ubicados en diferentes puntos geográficos, es necesario elaborar una propuesta idónea que dé respuestas a estas demandas y se adapte a los potenciales cambios de requerimientos [5].

El diseño de la MATe se alinea a las orientaciones de Alistair Cockburn [6] que desglosa una metodología en elementos. Se describen por lo tanto las funciones y los roles que ella requiere. Con este propósito se respetan las siguientes premisas:

- El ciclo productivo es corto, entregando un producto acotado aunque totalmente funcional cada 30 a 60 días. Este producto representa una Entrega de Valor Agregado (EVA) y está compuesta por al menos una historia del usuario, asumida esta última como la técnica utilizada para especificar los requisitos del software [7].
- Cada versión del producto se entrega debidamente testeada, por lo que cada ciclo productivo contempla el tiempo necesario para todos los tipos de pruebas, incluyendo las betas, es decir aquellas que involucran al cliente sin presencia del equipo de desarrollo.
- El trabajo se realiza a distancia en modalidad semi-sincrónica, con al menos 4 horas diarias de trabajo de programación conjunta, y el resto dedicado al testeo individual y a las pruebas de integración. Cada integrante del grupo de trabajo dispone del equipamiento necesario, tanto para el desarrollo de software como para las comunicaciones.
- El mejor lugar donde puede estar el cliente (de existir) es en su propio ámbito de trabajo. Eventualmente, y de ser posible, un miembro del equipo acude al domicilio del cliente para dialogar sobre los temas más sensibles.

A. Funciones y roles

En la metodología se reconocen los siguientes roles como indispensables para la ejecución de un proyecto de estas características, conformando todos ellos el "stakeholders" en el sentido aceptado del vocablo en el campo de los sistemas (aunque no todos los "stakeholders" conforman el equipo):

1). El dueño de la idea: En sí mismo constituye una interfaz que vincula al "cliente" con el "equipo". El cliente puede ser real y demandante (es decir que puede existir y desear una solución informática ajustada a determinadas funcionalidades y estándares de calidad) o, en el caso de concebir una aplicación sin cliente, es aquél que efectivamente ha tenido la idea y es capaz de cumplir con el rol de interfaz como si el cliente realmente existiera.

2). El facilitador: Es el líder de equipo. Comienza y termina las reuniones virtuales, determina el orden del día, enuncia los puntos acordados, fija posiciones con el dueño de la idea, gestiona los recursos y acuerda la extensión de cada EVA en función de lo propuesto por los desarrolladores.

3). El oráculo: Este actor gestiona toda la información concerniente al proyecto tal que cualquier integrante tenga un rápido acceso a la misma desde cualquier punto. Transforma el orden del día en una lista de eventos y genera eventos a medida que el orden del día es discutido.

4). Los desarrolladores: Los desarrolladores determinan la extensión de las tareas. Son quienes tienen a su cargo el desarrollo del código y su correspondiente testeo y depuración. Trabajan de a pares, compartiendo un escritorio virtual común, al menos cuatro horas diarias. El resto del tiempo testean y depuran el código.

5). El testeador: Este actor es el encargado de probar la integración de los componentes. Su función comienza cuando los desarrolladores concluyen las pruebas de unidad. Interactúa con el cliente durante las pruebas alfa y recibe la realimentación de las pruebas beta.

6). El cliente: El cliente es la base de la actividad comercial. Puede ser un comitente de existencia real, física, o jurídica, que encomienda el trabajo. En caso de que el equipo perciba una necesidad de mercado, éste pasa a asumir el rol del dueño de la idea.

Un integrante puede ejecutar más de un rol simultáneamente, desde su propio ámbito, incluyendo al cliente (de existir). Es probable que el dueño de la idea programe o que un programador utilice parte de su tiempo en oficiar de oráculo o bien que el testeador oficie de facilitador; dependiendo de las capacidades destacadas de cada miembro.

B. Herramientas

Existen en el mercado variados productos que podrían brindar un soporte completo a la metodología o hacerlo parcialmente, de disponer de algunas mejoras. La lista de aplicaciones que permiten el teletrabajo es extensa en términos de video chat o video conferencia, captura de escritorios remotos para el desarrollo de a pares y testeo de la aplicación, servidores colectivos de datos para la gestión de la documentación y convertidores de voz a texto para el registro de notas de las reuniones. Asimismo para facilitar la comunicación, la modalidad de interacción puede ser de a pares o de a grupos numerosos.

1). Evaluación comparativa de herramientas para trabajo de a pares: Previo a la definición del nuevo planteamiento se han efectuado pruebas sobre un conjunto de productos, tanto pagos como gratuitos, que ofrecen una potencial solución al problema que plantea el trabajo de abordaje distribuido. Para compartir las aplicaciones y el escritorio, y vincularse mediante video conferencia, se probaron las siguientes herramientas: Adobe Connect, TeamViewer, HangOut y Saros, tanto en pruebas individuales como combinadas, salvo con Adobe Connect.

Luego de las pruebas generales y la consecuente selección, se realizaron pruebas con TeamViewer y HangOut para luego evaluar el desempaño conjunto de HangOut y Saros, ambas herramientas corriendo al mismo tiempo en dos computadoras distintas. Una de ellas fue una máquina de escritorio, con altos recursos de hardware, y la otra fue una netbook con limitaciones en tanto a sus capacidades gráficas como de procesamiento.

Se procedió a probar la comunicación por medio de voz sobre ip en las circunstancias mencionadas. Como fruto de las experiencias puede destacarse que resulta más recomendable la utilización de Saros, para compartir la pantalla con el código de programación, y HangOut, que tiene la ventaja de indicar si las personas involucradas se encuentran conectados a Google+, Google talk o Gmail, para compartir la imagen y la voz. Por su parte se ha prescindido del Adobe Connect por no reunir los requerimientos mínimos que exige esta modalidad de trabajo y ser además relativamente costoso frente a las premisas descriptas con anterioridad. Bajo estas condiciones la programación en parejas opera con eficiencia, dado que la tecnología adoptada permite una comunicación fluida.

2). Evaluación comparativa de herramientas para el registro de las minutas: A los efectos de facilitar el trabajo distribuido, durante las reuniones periódicas, se pretende llevar registro de los diálogos mantenidos, en forma parcial o total, entre los stakeholders (integrantes del proyecto). La idea es adaptar un sistema automático de minutas para llevar el registro del diálogo mantenido en las reuniones. Se probaron para este fin, tres sistemas de captura de voz y su conversión a mp3: (a) Camtasia, (b) Voice Recorder y (c) Audacity, todos ellos igualmente recomendables.

3). Evaluación comparativa de herramientas para la conversión de las minutas a texto: La conversión se puede facilitar con Talktyper (www.taltyper.com) aunque no transcribe archivos mp3. Esta aplicación web es gratis y dispone de un micrófono que recoge la voz y la transcribe a texto. Para las transcripciones reconoce grabaciones hasta en 18 idiomas, incluyendo el español, el inglés y el francés. Otra herramienta muy útil es Dragon Dictation, una aplicación para móviles, compatible con dispositivos Apple y Android. En la actualidad se está trabajando con Notepad Lite, como conversor de voz a texto, especialmente diseñada para Android con las funcionalidades necesarias para poder trabajar con archivos de texto.

Como producto de las pruebas se adoptaron las herramientas que dieron respuesta a la funcionalidad y rendimiento de la MATe.

IV. Síntesis de la metodología

Tal como se ilustra en la Figura 1, y se describe a continuación, el proceso de cada EVA propone las siguientes fases:

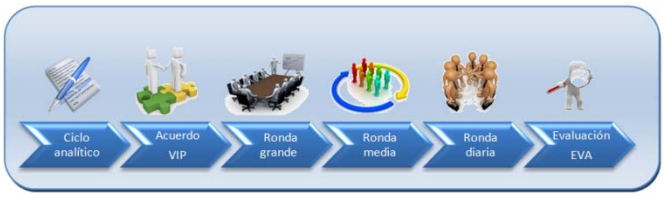


Figura 1: Fases de cada EVA

A. Ciclo analítico

Dada una idea, propia o de un cliente, se constituye un equipo de análisis formado por el cliente y los desarrolladores, conducido por el dueño de la idea. El Ciclo analítico representa la reunión donde se trata la formulación de la idea global, el listado de las funcionalidades esperadas y la generación de las historias de usuario que se describen en pocas frases y en lenguaje coloquial. Con estas historias se elabora un documento de precedencia de funciones donde se descomponen éstas en tareas y se fijan las prioridades.

B. Acuerdo VIP (Valor Individual Ponderado de las historias de cada EVA)

Este acuerdo se realiza una única vez por cada EVA, donde se define el "qué" se va a realizar. Se lleva a cabo en forma

previa a cada EVA e intervienen todos los involucrados, incluyendo al cliente de existir. La idea es capturar el espíritu de la aplicación que guiará el proceso de desarrollo, entendiendo "el espíritu de la aplicación" como el conjunto de anhelos que la justifica. En esta reunión el facilitador, en un todo de acuerdo con el dueño de la idea y con los desarrolladores, fijará el tiempo y el alcance de cada EVA. Una vez fijados estos parámetros, cada historia de usuario de la EVA es ponderada, por cada miembro productivo del equipo, a partir de la funcionalidad de la aplicación electrónica para la Metodología Ágil basada en Telecomunicaciones (e-MATe) que trabaja con distintos algoritmos. Esta técnica de planificación es una herramienta útil en la estimación del esfuerzo y de la complejidad de cada historia de usuario y que a su vez permite la estimación de los costos.

C. Ronda grande

En esta instancia se define el "cómo" se van a realizar las tareas, para lo cual cada par se auto-asigna sus tareas enmarcadas en la EVA. Una vez asignada una tarea no puede ser asignada a otro par, excepto que el par que la posee renuncie a la misma. Intervienen todos los miembros exceptuando al cliente.

D. Ronda media (semanal o quincenal)

Las Rondas medias se planifican los primeros días hábiles de cada semana e intervienen todos los involucrados, menos el cliente, dirigidos por el facilitador. Se revisa la marcha de las tareas y se ajustan a la realidad imperante.

E. Ronda diaria

Participa un par productivo y el facilitador en algún momento del día y debe ser muy breve. Se reportan avances y dificultades y se revisan las tareas por hacer hasta la siguiente ronda diaria. El facilitador replica esta reunión con cada par productivo.

F. Evaluación EVA

Concluido cada EVA se lleva a cabo una reunión que consta de:

1). Evaluación del producto: se reúnen todos los miembros del equipo de desarrollo con el cliente, o dueño de la idea, con el propósito de evaluar el EVA resultante.

2). Evaluación del proceso: se reúnen todos los miembros del equipo de desarrollo, exceptuando al cliente o dueño de la idea, con el propósito de evaluar el proceso, incluyendo las relaciones entre personas y las formas en que se podría mejorar en el próximo EVA. En esta oportunidad se registra en la e-MATe los tiempos reales que haya consumido cada tarea de cada miembro de equipo a los fines de generar la información referida al rendimiento de cada desarrollador.

La jornada de trabajo no debe superar las 8 hs diarias. Los pares productivos deben poder trabajar al menos 4 hs. conjuntas, por lo que sus locaciones geográficas y costumbres deben permitirlo, donde se trabaja en forma colaborativa y simultánea por captura de escritorio; mientras que el tiempo individual restante se dedica a la prueba y depuración.

V. FUNCIONALIDADES DE LA APLICACIÓN E-MATE

En esta etapa de la investigación se ha avanzado en el desarrollo de una aplicación que asiste al equipo de análisis en la implementación de la metodología propuesta. Durante este proceso de construcción se probó con rigor la MATe. Para este propósito el equipo estuvo conformado por desarrolladores radicados en diferentes países, con el mismo huso horario, confirmando el potencial de esta metodología para el trabajo distribuido. En este apartado se describen en forma sintética las funcionalidades de la aplicación.

El sistema e-MATe es una herramienta que ayuda a la organización y planificación de proyectos de desarrollo de sistemas, tanto para el uso de la metodología MATe como Scrum. El sistema brinda las herramientas para la administración de proyectos, administración de usuarios de sistema y las tareas propias de la metodología tales como historias de usuarios, la gestión de los EVA, la generación de estadísticas útiles para el cálculo de fechas de entregas, estimadas al finalizar los EVA, y la gestión del desarrollo.

e-MATe corre sobre plataforma móvil Android y web para permitir su fácil acceso y utilización. Entre las funcionalidades de e-MATe se encuentran:

A. Gestión de proyectos

e-MATe permite la creación de múltiples proyectos, registrando sus fechas de inicio y de culminación, las horas previstas y la cantidad de horas reales de desarrollo, entre otras variables, facilitando la administración de los roles de usuarios en cada proyecto.

B. Administración de usuarios de sistema

El alta de usuarios se da de manera general. Los roles son asignados individualmente en cada proyecto. Los usuarios pueden tener más de uno de los siguientes roles: (a) Dueño de la Idea, (b) Facilitador, (c) Oráculo, (d) Desarrollador, (e) Testeador y/o (f) Cliente.

C. Gestión de historias de usuarios

La aplicación registra las historias de usuarios, ordenándolas en una lista priorizada llamada Realizables. El dueño de la idea es el responsable de priorizarlas. La estimación de la complejidad, del grado de dificultad y de los costos de una historia se define mediante la propuesta del VIP (Valor Individual Promediado), donde los miembros del equipo debaten los resultados por medio de la comunicación propuesta para la MATe.

D. Gestión de EVA

Las EVA son creadas por el facilitador, tras una reunión de planificación. El facilitador deberá definir la duración y el alcance de la siguiente EVA. Para ello, la aplicación le sugiere la cantidad de historias de usuarios que podrían completar por medio del cálculo de la velocidad asumida del equipo y el ingreso de días laborales de la EVA.

F. Gestión del desarrollo

Las historias de usuario son tomadas de acuerdo a su orden de prioridad para una EVA. Solo el facilitador podrá quitar o agregar nuevas historias de usuario de una EVA.

Las historias son presentadas en el tablero de la e-MATe (Pizarra Digital) dividido en tres columnas según las referencias: (a) Historias pendientes, (b) Historias en proceso y (c) Historias finalizadas. A su vez cada historia se subdivide en un conjunto de tareas, definidas por los desarrolladores, cuya duración y dificultad son estimadas en el mencionado VIP.

La e-MATe presenta un segundo tablero para las tareas de cada historia. Los desarrolladores pueden adjudicarse una tarea como propia. Por su parte, cada desarrollador debe agregar la

cantidad de horas invertidas en el desarrollo de una tarea, tanto en forma individual como en pareja.

A partir de la posición de las tareas de una misma historia, dentro del tablero, se calcula la posición y el estado de la historia en la Pizarra Digital. En el caso de que todas las tareas de una historia se encuentren Pendientes, la historia estará Pendiente. Si alguna tarea estuviera en Desarrollo, la historia estará en Desarrollo. Si no hubiera tareas Pendientes o en Desarrollo, la historia se encontraría Finalizada. Si alguna historia presentara un impedimento para su realización, el desarrollador podrá marcar la tarea como Impedida y describir sus causas, lo que se mostrará en la e-MATe por medio de una señal en la historia. Es responsabilidad del facilitador solucionar el impedimento y reactivar la tarea.

Una vez que una historia esté completa deberá ser revisada y aprobada por el dueño de la idea a los efectos de ser ubicada en el estado Finalizada.

La e-MATe acompaña la evolución de los procesos mediante la generación de una gráfica de avance que exhibe el volumen del trabajo realizado, el correspondiente al trabajo faltante y una proyección del plazo restante para completar las historias de la EVA. Asimismo, para cada uno de los desarrolladores, la e-MATe almacena datos vinculados con las horas trabajadas, tanto en forma individual como en pareja y la dificultad promedio de las historias y de las tareas en las que trabaja. Partiendo de estos datos, la aplicación genera información referida al rendimiento del desarrollador, según la cantidad y complejidad de cada tarea asignada. De esta manera el facilitador puede obtener, en forma precisa, una reseña sobre el desempeño de los responsables de cada historia y sobre el estado de avance del proyecto en cuestión.

G. Documentación

En una historia, la e-MATe permite incluir requisitos no funcionales y además admite crear notas retrospectivas que podrán ser consideradas al momento de revisar los resultados de una EVA. Asimismo es factible crear comentarios en las distintas tareas o historias. Los distintos usuarios pueden utilizar estos comentarios para responder a particularidades que requieran una explicación.

La versión web de la aplicación permite exportar el conjunto de fichas de historias de usuarios de un proyecto, así como las gráficas de avance de las distintas EVA y demás información relacionada con las mismas. Una vez completadas las fichas, esta funcionalidad permite exportar la información en un documento más ágil de entregar o evaluar por los interesados o por agentes externos.

REFERENCIAS

[1] Letelier, P. y Penadés, M. C. Metodologías ágiles para el desarrollo de software: eXtreme Programming (XP). Universidad Politécnica de Valencia. Documento recuperado de http://www.willydev.net/descargas /masyxp.pdf (2006)

[2] Blanco, P., Camarero, J., Fumero, A., Werterski, A., Rodríguez, P. Metodología de desarrollo ágil para sistemas móviles: Introducción al desarrollo con Android y el iPhone. Universidad Politécnica de Madrid. Documento recuperado de http://www .adamwesterski.com/wp-content/files/docsCursos /Agile_doc_TemasAnv.pdf (2009)

[3] Ableson, Frank y Sen, Robi. Android: Guía para Desarrolladores (2º Ed.) - Madrid: Anaya Multimedia (2011)

[4] Scalone, F. Estudio comparativo de los modelos y estándares de calidad del software. Tesis de Maestría. Recuperado de http://laboratorios.fi.uba.ar/lsi/scalone-tesis-maestria-ingenieria-en-calidad.pdf (2006)

[5] [5] Gamma E., Helm, R., Johnson, R. y Vlissides, J. Patrones de Diseño. Madrid: Pearson Educación (2003)

[6] Cockburn, Alistair, Surviving Object Oriented Projects, Addison- Wesley Object Technology Series (1998)

[7] Calderón, Amaro, Valverde Rebaza, Sarah Dámaris y Carlos, Jorge. Metodologías Ágiles. Facultad de Ciencias Físicas y Matemáticas, Universidad Nacional de Trujillo, Perú Recuperado de http://www.seccperu.org/files /Metodologias%20Agiles.pdf (2007)



Marisa Cecilia Tumino. Ingeniera en Recursos Hídricos, Analista en Informática Aplicada, Magister en Ciencias Computacionales, Doctora en Educación con énfasis en Administración educativa. Desempeño profesional en (a) Instituto Francisco Ramos Mejía, (b) Universidad Adventista del Plata, (c) Instituto Juan Bautista Alberdi- Misiones, (d) Universidad de Montemorelos, México, (e) Herbert Fletcher University, Puerto Rico y (f) Universidad de Chillán, Chile.



Juan Bournissen. Doctorando en Tecnologías Educativas: E-learning, y Gestión del Conocimiento en la Universidad de Islas Baleares, España. Master en Ingeniería del Software obtenido en la Universidad Politécnica de Madrid. Magíster en Ingeniería del software obtenido en el Instituto Tecnológico de Buenos Aires, ITBA. Especialista en Entornos Virtuales del Aprendizaje. Profesor Universitario en Sistemas de Información. Ingeniero en Sistemas de Información. Analista Universitario en Sistemas. Profesor universitario de grado y posgrado en la modalidad presencial y a distancia. Administrador de plataformas virtuales en instituciones estatales y privadas. Formador de formadores en e-learnig en Argentina y en varios países de Latinoamérica. Director de centro de cómputos. Asesor informático en sistemas de información y en tecnologías educativas y e-lerning.



Claudio José Bracalenti: Ingeniero en Construcciones (UTN FRSF 1984). Calculista en Agua y Energía eléctrica 1979-1980. Programador en AyE entre 1981 – 1984. Ingeniería de Sistemas en AyE entre 1985 y 1992. Docente de la UTN FRSF desde 1984 habiendo enseñado en varias cátedras. Actual Profesor Titular en Diseño de Sistemas y Proyecto Final desde 1992 a la fecha en la carrera de Ingeniería en Sistemas de Información de la UTN FRSF. Coordinador de Informática en la Universidad Nacional del Litoral entre 1995 – 2000, Consejero Departamental entre 1992 y 2000 y Académico entre 1992 y 2002 en la UTN FRSF. Docente de la Universidad Adventista del Plata desde 1996 habiendo enseñado en varias cátedras. Actual Profesor Titular de Ingeniería del Software II, Administración de Recursos Informáticos e Informática aplicada desde el 2001 a la fecha en la carrera de Licenciatura en Sistemas de Información de la UAP. . Subsecretario de Planeamiento de la UTN FRSF entre 2000-2002.



Eric Germán Schlemper: Argentino. Estudios en curso: Licenciatura en Sistemas, Universidad Adventista del Plata (2008 - Presente). Ocupación Actual: Desarrollo de aplicaciones con orientación médica, Sanatorio Adventista del Plata (2013). Intereses: Diseño y desarrollo de aplicaciones

distribuidas. Informática Biomédica. Desarrollo de aplicaciones con metodologías agiles.



Silvio Rodrigo Kucharski Zuliani: Estudiante del último año de Licenciatura en Sistemas de Información en la Universidad Adventista del Plata (UAP). Ponente en el XV Workshop de Investigadores en Ciencias de la Computación (WICC), en Paraná, Entre Ríos, con el título "Metodología Ágil basada en Telecomunicaciones". Desarrollador de una "Nueva metodología ágil para dispositivos móviles".

Investigación en Progreso: Método de Evaluación Dinámica de Planes en Sistemas Inteligentes Autónomos

Ezequiel González[1,2]
1. Programa de Maestría en Ingeniería de Sistemas de Información.
Escuela de Posgrado, Facultad Regional de Buenos Aires. Universidad Tecnológica Nacional. Argentina.
2. Laboratorio de Investigación y Desarrollo en Sistemas de Inteligencia Artificial.
Grupo de Investigación en Sistemas de Información. Universidad Nacional de Lanús. Argentina.
ezequielgonzalez81@gmail.com

***Resumen* — El modelo LOPE es un sistema inteligente autónomo con aprendizaje basado en formación y ponderación de teorías. En los últimos años, se han elaborado varias modificaciones al modelo que han mejorado el rendimiento de su aprendizaje y de su planificación. Sin embargo, hay ciertos aspectos que aún no han sido abordados y cuyo diseño e implementación se cree incrementaría la convergencia de aprendizaje del modelo. Ellos son: (a) el proceso de aprendizaje sólo se produce en la fase de observación, perdiendo la oportunidad de aprender a partir de la evaluación del resultado de los planes ejecutados, (b) el índice utilizado para medir la calidad de los planes antes de ser ejecutados es un parámetro fijo, dejando abierta la posibilidad de que se ejecute una alta cantidad de planes condenados al fracaso. Este proyecto se propone desarrollar modificaciones o extensiones al modelo con el fin de mejorar estos aspectos e incrementar la curva de aprendizaje del sistema.**

***Palabras Clave* — sistemas inteligentes autónomos, aprendizaje no supervisado, planificación, evaluación de la planificación, formación de teorías, ponderación de teorías.**

I. JUSTIFICACIÓN DE LA PROPUESTA

El modelo LOPE [8] [9] es un sistema inteligente autónomo (SIA) con aprendizaje basado en formación y ponderación de teorías. Puede ser descrito como un robot de exploración que percibe el entorno a través del sistema sensor y registra teorías locales a partir de la situación previa, la acción ejecutada y la situación resultante. Dichas teorías son utilizadas para la construcción de planes que le permitirán alcanzar sus propios objetivos. En los últimos años, se han elaborado varias modificaciones al modelo con el fin de mejorar el rendimiento de su aprendizaje y de su planificación. En [10] se incorporó una arquitectura multiagente que permite el intercambio de teorías; en [15] se utilizó un sistema multiagente pero en este caso se definieron distintos perfiles de agentes, cada uno de los cuales con un determinado método de adquisición y transmisión de conocimiento; en [17] se propuso un mecanismo de ponderación de planes distinto; por último, en [24] se tomó como marco de trabajo el algoritmo de ponderación elaborado por López pero se propuso la aplicación de algoritmos genéticos para mejorar el rendimiento.

A pesar de que en cada una de las soluciones propuestas, el rendimiento del sistema mejoró, hay ciertos aspectos del modelo que aún no han sido abordados y cuyo diseño e implementación es el objetivo de este proyecto de investigación. En primer lugar, es importante destacar que tanto en el modelo LOPE original como en las variantes mencionadas, el proceso de aprendizaje sólo se produce dentro de la fase de observación. En esta etapa, de acuerdo al resultado de lo percibido por el sistema sensor, el sistema refuerza teorías o pondera y crea teorías mutantes. Ahora bien, en el caso que exista un plan en ejecución y que uno de sus pasos no haya logrado la situación esperada, el sistema sólo se limita a abortar el plan y a devolver el control al planificador; perdiendo la oportunidad de incrementar su aprendizaje a partir del resultado de los planes. Por lo tanto, dado que un plan consiste en una concatenación de teorías, generar un mecanismo que refuerce o castigue las teorías involucradas en ellos, de acuerdo a su éxito o fracaso, mejoraría sin dudas el proceso de aprendizaje del modelo.

En segundo lugar, se observa una oportunidad de mejora en el proceso de evaluación de calidad de los planes a ejecutar, ya que el parámetro que mide la confiabilidad de los planes es un valor estático a lo largo de todo el ciclo de vida del agente. Es decir, a la hora de decidir si un plan será llevado a cabo o no, el sistema evalúa si su probabilidad de éxito es mayor o igual al umbral de confiabilidad. En caso afirmativo, el sistema inicia la ejecución del plan, de lo contrario, intenta armar un nuevo plan. Sin embargo, cabe preguntarse, ¿qué sucedería si como diseñadores del sistema observáramos que una gran cantidad de planes han fracasado? En ese caso, ¿no sería conveniente que el umbral de confiabilidad se incremente para evitar que se ejecuten planes condenados al fracaso? El autor entiende que sí, y debido a ello, propone la creación de un mecanismo que permita configurar un índice de confiabilidad dinámico.

II. ESTADO ACTUAL DEL CONOCIMIENTO SOBRE EL TEMA

Con base en la reseña formulada en la tesis doctoral del Dr. Jorge Ierache [14], se presenta una breve descripción de los sistemas inteligentes autónomos (sección II.A), de los SIA con aprendizaje basado en formación y ponderación de teorías (sección II.B) y de los SIA con aprendizaje basado en intercambio de teorías (sección II.C). Completando la reseña del autor precitado, se incluye el mecanismo de ponderación basado en la productoria de estimadores de probabilidad de éxito de acciones (sección II.D) y el método de aplicación de algoritmos genéticos al modelo en cuestión (sección II.E).

A. Sistema Inteligente Autónomo

Uno de los puntos involucrados en el problema de la modelización de Sistemas Inteligentes [5] [6] [7] es lograr una base axiomática que describa formalmente los fenómenos que tienen lugar en este tipo de sistemas. Esta descripción formal apunta a proporcionar un instrumento para clasificar, medir y calcular en el campo de la inteligencia. Formalmente, no es relevante la clasificación en natural o artificial. El propósito del trabajo es abstraer los rasgos comunes, si los hay, de todos los procesos inteligentes. Luego, clasificar como inteligentes a los sistemas capaces de dar lugar a procesos inteligentes.

Un rasgo comúnmente asociado con la inteligencia es la capacidad de adquirir nuevos conocimientos. Esto se manifiesta en los procesos de aprendizaje, que aceptan ser descritos en términos de asimilación e incorporación de información extraída del contexto. Una forma de adquirir conocimiento nuevo es el llamado "método del ensayo-error"; esta técnica permite descubrir leyes simples cuya verdad se deduce a partir de la experiencia. En la teoría presentada por los autores citados, esta adquisición de conocimiento está centrada alrededor de la asimilación de experiencias, siendo las leyes empíricas las unidades de experiencia.

Los Sistemas Inteligentes tienen objetivos, que consisten en acceder a una situación que les conviene. Están capacitados además para elegir sus acciones según tales objetivos y son capaces de aprender qué acción es útil efectuar en cada situación en relación a los mismos. La situación es el conjunto de los rasgos esenciales del estado de las cosas, en relación a los objetivos del sistema. Se elabora sobre la base de todas las entradas sensoriales del momento y sobre su conceptualización. Sobre la base de esta modelización se elige cada acción. Para lograr sus objetivos, los Sistemas Inteligentes actúan, y para poder elegir acciones adecuadas deben contar con una memoria en la cual archivan sus experiencias.

Una unidad de experiencia se compone (por lo menos) de la situación vivida, la acción realizada, la situación resultante y el hecho de que las consecuencias de la acción hayan sido beneficiosas o no para lograr el objetivo. Este beneficio, o la falta del mismo, se traduce en utilidad resultante. La decisión sobre la acción que conviene realizar se toma en función de las experiencias acumuladas, si es que están en relación con las circunstancias actuales (pueden ser tanto experiencias directas del sistema como también experiencias conocidas a través de lo que se verificó en otros). Si en lo archivado como experiencia tal relación existe y la acción elegida en aquél entonces resultó beneficiosa, habrá una tendencia de elegir nuevamente esa misma acción u optar por alternativas distintas si la acción resultó perjudicial.

Cuando se trata de una situación nueva, esto es, no existe experiencia previa de la misma, se efectúan acciones razonadas guiándose por los resultados obtenidos en actuaciones anteriores, o si no, por intuición, instinto o incluso al azar. Frente a situaciones conocidas, los Sistemas Inteligentes tienden a desarrollar una actuación que (por experiencia) consideran óptima (no necesariamente es la óptima). Esta tendencia se denomina hábito. Un mal hábito se da cuando el sistema persiste en un cierto actuar, aun cuando éste ya no corresponde a la situación.

B. Sistema Inteligente Autónomo con aprendizaje basado en formación y ponderación de teorías

El sistema puede ser descrito como un robot de exploración que percibe el entorno a través del sistema sensor, registra la situación, y arma una teoría local con la situación previa y la acción ejecutada [8] [9]. Si la teoría local es igual a alguna teoría registrada, ésta se refuerza. Si es similar, se registra la teoría local, se pondera y se crean teorías mutantes. Si existe un plan en ejecución, se verifica que la situación obtenida sea la esperada; si no ocurre esto, se aborta el plan y el control es devuelto al planificador. Si no existe un plan en ejecución, el planificador genera uno, lo envía al ponderador y mediante un criterio heurístico (optimizar las acciones a ejecutar por el sistema, sobre la base del umbral adoptado para la ponderación del plan), se determina si el plan es aceptable. En caso afirmativo, se pasa el control al controlador de planes en ejecución, cuya función es determinar la siguiente acción a ser ejecutada y establecer si las situaciones obtenidas son las situaciones esperadas. Si el plan no es confiable, se genera un nuevo plan. La arquitectura del sistema puede ser descrita mediante el esquema que se presenta en la Figura 1.

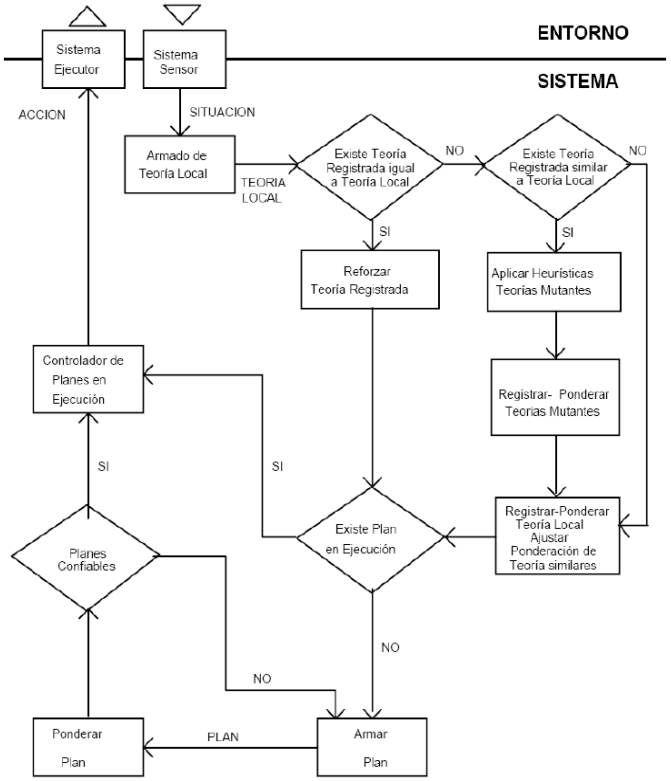


Fig. 1. Esquema del Sistema Inteligente Autónomo con aprendizaje basado en formación y ponderación de teorías

C. Sistema Inteligente Autónomo con aprendizaje basado en Intercambio de teorías

Esta arquitectura de sistema inteligente autónomo también percibe el entorno a través del sistema sensor, pero antes de realizar cualquier acción, se pregunta si es necesario intercambiar operadores con otro sistema inteligente autónomo [10]. Este proceso se lleva a cabo mediante un módulo de intercambio de operadores. Luego, se registra la situación percibida del entorno y arma una teoría local con la situación previa y la acción ejecutada. En la figura 2 se presenta la arquitectura del sistema, donde se observa su interacción con el entorno y el funcionamiento general de los módulos de

aprendizaje, planificación, ponderación, control e intercambio de operadores.

Si la teoría local es igual a alguna teoría registrada, ésta se refuerza, si no existe una teoría igual pero existen similares, éstas se ponderan y se generan teorías mutantes las cuales son registradas y ponderadas de la misma forma. Por último (luego del proceso de generar teoría mutantes o si no existen teorías similares) se incorpora la teoría local y se pasa el control al subsistema controlador.

Si existe un plan en ejecución, se verifica que la situación obtenida sea la esperada; si no ocurre esto, se aborta el plan y el control es devuelto al planificador. Si no existe un plan en ejecución, el planificador genera uno, lo envía al ponderador y mediante un criterio heurístico, se determina si el plan es aceptable. En caso afirmativo, el controlador de planes en ejecución determina la siguiente acción a ser ejecutada, la cual es pasada a la plataforma para que ésta la aplique en el entorno.

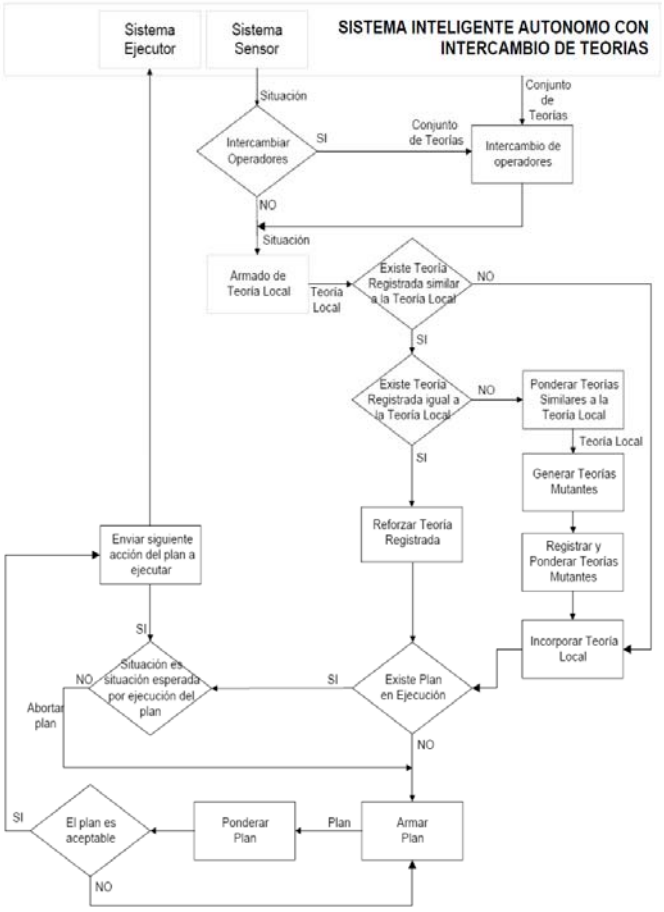


Fig. 2. SIA con aprendizaje basado en formación, ponderación e intercambio de teorías

D. Método de Ponderación basado en la productoria de estimadores de probabilidad de éxito de acciones en SIA

En [17] se propone un nuevo algoritmo de ponderación de planes con el objetivo de mejorar el rendimiento del sistema (porcentaje de planes exitosos). Para estimar la probabilidad de éxito de los planes, el método clásico de ponderación se basa en la teoría de autómatas estocásticos, ya que el conocimiento que posee el sistema en un momento dado (el conjunto de teorías), puede ser visto como un modelo de cómo reaccionará el entorno frente a las acciones que ejecuta el sistema. O sea, si se consideran todos los estados en que puede encontrarse el autómata cuando recibe una entrada, puede definirse una matriz de transición que contenga la probabilidad de éxito del plan.

La primera modificación que lleva adelante López es dar una nueva representación para la definición de un Plan. Mientras que en el modelo LOPE original, un plan (P) tiene la estructura definida en la Ecuación 1:

$$P_{ij} = A_1 \text{ o } A_2 \text{ o } \dots \text{ o } A_r \quad (1)$$

donde A_1, A_2 y A_r identifican a cada una de las acciones necesarias para alcanzar la situación esperada; con la nueva propuesta, el Plan se define de acuerdo a la Ecuación 2:

$$P^*_{ij} = (s_{i1},a_1,s_{f1},P_1,K_1) \text{ o } (s_{i2},a_2,s_{f2},P_2,K_2) \text{ o } \dots \text{ o } (s_{ir},a_r,s_{fr},P_r,K_r) \quad (2)$$

donde s_i y s_f representan a la situación inicial y final respectivamente, a identifica a la acción ejecutada, K es la cantidad de veces que una teoría fue utilizada y P el número de veces que esa teoría fue utilizada con éxito.

La diferencia entre ambas definiciones radica en que, si bien ambas representan un mismo plan, la segunda contiene mayor información, ya que expresa el plan como una composición de las r teorías en que se basó su construcción, haciendo explícitas las respectivas situaciones iniciales y finales esperadas a cada paso de la ejecución del plan.

En segundo lugar, mientras que el modelo clásico se basa en la matriz de transición para calcular la probabilidad de éxito de un determinado plan, en el método propuesto, la probabilidad estimada de que el plan P^*_{ij} aplicado a la situación s_i resulte en la situación s_j, se obtiene calculando el producto de números escalares definido en la Ecuación 3:

$$P_{exito} = P_1/K_1 \,.\, P_2/K_2 \dots P_r/K_r \quad (3)$$

Es decir, en este último caso, la probabilidad de éxito del plan es igual a la productoria de las probabilidades de éxito de cada una de las acciones por separado.

E. Algoritmos genéticos aplicados a los Sistemas Inteligentes Autónomos

Un Algoritmo Genético (AG) es una técnica de búsqueda basada en la teoría de la evolución de Darwin, en el cual los individuos más aptos de una determinada población son los que sobreviven, ya que pueden adaptarse más fácilmente a los cambios que se producen en su entorno. Algunas de sus características son: (i) requieren muy poca información específica del problema, (ii) resuelven en forma rápida y eficiente problemas con espacio de búsqueda muy grandes, (iii) alta dimensionalidad de la función de aptitud y iv) funciones de aptitud no lineales [11].

En [24] el AG es implementado en el SIA del siguiente modo. Una vez que este percibe una nueva situación, se procede al armado de la teoría local. Si existe al menos una teoría similar y no hay teorías iguales a la nueva teoría local, se ponderan las teorías similares, se generan las teorías mutantes, se registran y ponderan las teorías mutantes y se incorpora la teoría local. Luego, si se sucedió una cantidad mínima de ciclos sin aplicar AG y se iguala o supera una cantidad específica de teorías, se aplica el AG al conjunto de teorías del SIA.

El AG acelera los tiempos de aprendizaje del SIA, ya que al combinarlo con otras estrategias, provoca un aumento en la cantidad de teorías que el sistema adquiere. También se observa un gran aumento de teorías cuando se combina mutación, intercambio, ponderación y AG, en comparación con los resultados obtenidos sin AG.

III. Objetivo del proyecto de investigación

El objetivo de este proyecto de investigación es explorar soluciones alternativas o extensiones al modelo LOPE [8] [9], de forma tal de resolver las debilidades previamente descritas y que aún no han sido abordadas por los autores citados.

De acuerdo a lo descrito en la sección I, el modelo presenta dos oportunidades de mejora para optimizar el proceso de aprendizaje del sistema.

La primera de ellas está vinculada a la falta de aplicación de un método que refuerce los planes exitosos y/o castigue a los planes que han fracasado, evitando de este modo que se incrementen las instancias de aprendizaje del sistema. Este trabajo buscará desarrollar un algoritmo por el cual, los resultados de los planes ejecutados sean aprovechados para mejorar la convergencia del modelo.

La segunda oportunidad de mejora se refiere a la característica estática del umbral de confiabilidad, siendo ésta propiedad una posible limitación para el perfeccionamiento del proceso de evaluación de planes. En línea con esto, se explorará la posibilidad de desarrollar un algoritmo alternativo que permita la configuración de un índice de confiabilidad dinámico.

IV. Metodología de desarrollo

Para construir el conocimiento del presente proyecto de trabajo, se seguirá un enfoque de investigación clásico [21] [4] con énfasis en la producción de tecnologías [23]; identificando métodos y materiales necesarios para desarrollar el proyecto.

A. Métodos

A continuación se definen los métodos que se llevarán a cabo en el presente proyecto de investigación. Ellos son:

- Revisiones sistemáticas: Las revisiones sistemáticas [2] de artículos científicos siguen un método explícito para resumir la información sobre determinado tema o problema. Se diferencia de las revisiones narrativas en que provienen de una pregunta estructurada y de un protocolo previamente elaborado.
- Prototipado evolutivo experimental (método de la Ingeniería): El prototipado evolutivo experimental [3] consiste en desarrollar una solución inicial para un determinado problema, generando su refinamiento de manera evolutiva por prueba de aplicación de dicha solución a casos de estudio (problemáticas) de complejidad creciente. El proceso de refinamiento concluye al estabilizarse el prototipo en evolución.

B. Materiales

A continuación se detallan los materiales que se utilizarán para el desarrollo del proyecto de investigación:

- Formalismos de modelado conceptual usuales en la Ingeniería de Software [22] y enfoques recientes [12].
- Modelos de Proceso usuales en Ingeniería de Software [13] [1] [19].
- Ambientes de simulación de plataformas robóticas móviles [16].
- Entorno de Programación JAVA.
- Paquetes estadísticos usuales para el análisis de los resultados experimentales [20].

C. Metodología

Para alcanzar los Objetivos trazados se propone: (i) realizar una investigación documental exploratoria sobre algoritmos asociados a los procesos de planificación en sistemas inteligentes autónomos, (ii) identificar casos de estudio y casos de validación, (iii) desarrollar mediante la metodología de prototipado evolutivo un algoritmo por el cual, los resultados de los planes ejecutados sean aprovechados para el proceso de aprendizaje, (iv) desarrollar mediante la metodología de prototipado evolutivo un algoritmo que permita la configuración de un índice de confiabilidad dinámico, (v) realizar pruebas de concepto en los casos de estudio y casos de validación identificados, que validen los algoritmos propuestos y (vi) realizar una simulación para analizar el funcionamiento de la arquitectura del sistema inteligente autónomo sin los algoritmos propuestos (a), con el algoritmo que introduce la retroalimentación a partir de los resultados de los planes ejecutados (b), con el algoritmo que genere un índice de confiabilidad dinámico (c) y con ambos algoritmos juntos (d).

V. Condiciones institucionales para el desarrollo del proyecto de investigación

Este proyecto articula líneas de investigación en el área de Sistemas Inteligentes Autónomos del Laboratorio de Investigación y Desarrollo en Sistemas de Inteligencia Artificial (LIDSIA UNLa), con radicación en el Departamento de Desarrollo Productivo y Tecnológico de la Universidad Nacional de Lanús. Las líneas de investigación del área cuentan con financiamiento de la Secretaría de Ciencia y Técnica de la misma Universidad.

VI. Bibliografía

[1] ANSI/IEEE (2007). “Draft IEEE Standard for software and system test documentation”. ANSI/IEEE Std P829-2007.

[2] Argimón, J. (2004). “Métodos de Investigación Clínica y Epidemiológica”. Elsevier España, S.A. ISBN 9788481747096.

[3] Basili, V. (1993). “The Experimental Paradigm in Software Engineering”. En Experimental Software Engineering Issues: Critical Assessment and Future Directions (Ed. Rombach, H., Basili, V., Selby, R.). Lecture Notes in Computer Science, Vol. 706. ISBN 978-3-540-57092-9.

[4] Creswell, J. (2002). “Educational Research: Planning, Conducting, and Evaluating Quantitative and Qualitative Research”. Prentice Hall. ISBN 10: 01-3613-550-1.

[5] Fritz, W. (1984). “The Intelligent System.” SIGART Newsletter, 90: 34-38. ISSN 0163-5719.

[6] Fritz, W. (1992). “World view and learning systems”. Robotics and Autonomous Systems 10(1): 1-7. ISSN 0921-8890.

[7] Fritz, W., García Martínez, R., Rama, A., Blanqué, J., Adobatti, R. y Sarno, M. (1989). “The Autonomous Intelligent System”. Robotics and Autonomous Systems, 5(2):109-125. ISSN 0921-8890.

[8] García-Martínez, R. y Borrajo, D. (1997). “Planning, Learning and Executing in Autonomous Systems”. Lecture Notes in Artificial Intelligence 1348: 208-210. ISBN 978-3-540-63912-1.

[9] García-Martínez, R. y Borrajo, D. (2000). “An Integrated Approach of Learning, Planning and Executing”. Journal of Intelligent and Robotic Systems 29(1): 47-78. ISSN 0921-0296.

[10] García-Martínez, R., Borrajo, D., Britos, P. y Maceri, P. (2006). “Learning by Knowledge Sharing in Autonomous Intelligent Systems”. Lecture Notes in Artificial Intelligence, 4140: 128-137. ISBN 978-3-540-45462-5.

[11] García-Martínez, R., Servente, M. y Pasquini, D. (2003). “Sistemas Inteligentes” (pp. 149-280). Buenos Aires: Editorial Nueva Librería. ISBN Nº 987-1104-05-7.

[12] Hossian, A. (2012). “Modelo de Proceso de Conceptualización de Requisitos” (Tesis Doctoral en Ciencias informáticas). Facultad de Informática. Universidad Nacional de La Plata.

[13] IEEE (1997). “IEEE Standard for Developing Software Life Cycle Processes. IEEE Std 1074-1997” Revision of IEEE Std 1074-1995; Replaces IEEE Std 1074.1-1995.

[14] Ierache, J. (2010). “Modelo de ciclo de vida para el aprendizaje basado en compartición de conocimientos en sistemas autónomos de robots”. (Tesis Doctoral en Ciencias Informáticas). Facultad de Informática. Universidad Nacional de La Plata.

[15] Ierache, J., García-Martínez, R. y De Giusti, A. (2008), “Learning Life-Cycle in Autonomous Intelligent Systems”. Artificial Intelligence in Theory and Practice II, ed. M. Bramer, (Boston: Springer), pp 451- 455, ISSN 1571-5736.

[16] Khepera III (2010). “Robot Khepera-III”, publicado en: http://www.k-team.com/mobile-robotics-products/khepera-iii. Página vigente al 05/11/2013.

[17] López, D., Merlino, H., Ierache, J. y García Martínez, R. (2008). “A Method for Pondering Plans in Autonomous Intelligent Systems”. Anales V Workshop de Inteligencia Artificial Aplicada a la Robótica Movil. Pág. 98-104. ISBN 978-987-604-100-3.

[18] Mondada, F., Franzi, E. y Guignard A. (1999). “The Development of Khepera”. First International Khepera Workshop, Paderborn, HNI-Verlagsschriftenreihe, Heinz Nixdorf Institut 64.

[19] Oktaba, H., Garcia, F., Piattini, M., Ruiz, F., Pino, F. y Alquicira, C. (2007). “Software Process Improvement: The Competisoft Project”. IEEE Computer, 40(10): 21-28. ISSN 0018-9162.

[20] OriginLab (1999). “Origin 6.0”, publicado en: http://www.originlab.com/index.aspx?go= PRODUCTS/Origin. Página vigente al 05/11/2013.

[21] Riveros, H. y Rosas, L. (1985). “El Método Científico Aplicado a las Ciencias Experimentales”. México: Editorial Trillas. ISBN 96-8243-893-4.

[22] Rumbaugh, J., Jacobson, I. y Booch, G. (1999). “The Unified Modeling Language, Reference Manual”. Addison Wesley, ISBN-10: 02-0130-998-X.

[23] Sabato J, Mackenzie M. (1982). “La Producción de Tecnología: Autónoma o Transnacional”. Instituto Latinoamericano de Estudios Transnacionales - Technology & Engineering. ISBN 9789684293489.

[24] Steinhilber, R., García-Martínez, R. y Kuna, D. (2009). “Mutación de Teorías en Sistemas Inteligentes Autónomos Basada en Algoritmos Genéticos”. Proceedings VII Campeonato de Fútbol de Robots y Workshop de Sistemas Autónomos de Robots. Pág. 24-33. ISBN 978-950-9474-45-1.



Ezequiel González. Es Licenciado en Economía por la Universidad Torcuato Di Tella. Es Candidato del Programa de Magister en Ingeniería de Sistemas de Información de la Escuela de Postgrado de la Facultad Regional Buenos Aires de la Universidad Tecnológica Nacional. Es Investigador Tesista del Laboratorio de Investigación y Desarrollo en Sistemas de Inteligencia Artificial del Grupo de Investigación en Sistemas de Información de la Universidad Nacional de Lanús. Es Consultor Independiente en Procesos y Sistemas de Información. Sus áreas de interés son Aprendizaje y Planificación en Sistemas Inteligentes Autónomos y sus aplicaciones al ámbito empresarial.